课程笔记

需求和架构（上）：AI 时代的软件工程杠杆

南京大学 2026 生成式软件工程 · 第 06 讲 · 从需求工程到事件溯源

lecture-to-notes

2026 年 9 月 22 日



视频作者/频道：绿导师原谅你了（B 站）
发布日期：2026-09-22
视频时长：1 小时 36 分 50 秒
视频链接：https://www.bilibili.com/video/BV1Rch76WEfQ/

目录

1 引子：写代码变便宜之后，什么变贵了 3

1.1 上一讲留下的三个关键词 . . . 3
1.2 “软件不软”：逻辑制品容不下细微差错 . . . 4
1.3 需求、产品、架构：写代码之前的三件事 . . . 4
1.4 一段事故插曲：可观测性也是债 . . . 6
1.5 本章小结 . . . 6

2 需求工程：从一个真实的教务系统说起 6

2.1 一个“史诗级需求爆炸”的样本 . . . 6
2.2 需求是叠加出来的 . . . 7
2.3 本章小结 . . . 8

3 把需求工程交给模型：一份真实的需求说明书 8

3.1 “用魔法打败魔法”：先调研，再实现 . . . 8
3.2 需求说明书写了什么 . . . 8
3.3 只读需求说明书，能做出一个系统吗 . . . 10
3.4 本章小结 . . . 11

4 需求的另一半：人、领域知识与不确定性 11

4.1 为谁设计：同理心与用户画像 . . . 11
4.2 领域知识与行业壁垒 . . . 12
4.3 案例：培养方案、学分与“最后的暗箱操作” . . . 12
4.4 一次亲手制造的事故：把缺课次数改成 -1 . . . 14
4.5 没有银弹 . . . 15
4.6 本章小结 . . . 16

5 架构：控制复杂性的手艺 16

5.1 架构不是玄学 . . . 16
5.2 最小的一段代码也有架构 . . . 17
5.3 `getopt`：把复杂性隔离成一个解析状态 . . . 18
5.4 本章小结 . . . 19

6 十行的架构：非递归汉诺塔的四种表达 19

6.1 为什么大家都写不出来 . . . 19
6.2 第一种：GPT-6 的数学规律 . . . 19
6.3 第二种：DeepSeek 的任务栈 . . . 20
6.4 第三种：把 C 语言的小步语义写成状态机 . . . 20
6.5 架构能力从哪里来 . . . 24
6.6 本章小结 . . . 24

7 两千行的架构：在 Excel 里生成二维码 24

7.1 一个“需求非常明确”的项目 . . . 24
7.2 跳出问题看问题：这其实是一个编译问题 . . . 25

7.3 中间语言长什么样 . . . 26
7.4 一份中间语言，两种执行方式 . . . 27
7.5 本章小结 . . . 29

8 五十万行的架构：AI 交出来的教务系统 29
8.1 先用一句话问它：这是什么架构 . . . 29
8.2 它自己都做对了什么：生产级横切设计 . . . 29
8.3 它的扩展性好在哪里，坏在哪里 . . . 30
8.4 它还差什么：88 条需求与“横切面切片” . . . 31
8.5 本章小结 . . . 31

9 CRUD 架构：把世界当作当前状态的投影 31
9.1 一个几乎无处不在的世界观 . . . 31
9.2 范式化：把冗余从表里赶出去 . . . 32
9.3 本章小结 . . . 33

10 CRUD 为什么会死：历史、规则与决定 33
10.1 “当前状态”里其实混着三种东西 . . . 33
10.2 有损投影：系统外的事情已经发生了 . . . 34
10.3 语义也会变：计算机系变成计算机学院 . . . 35
10.4 系统一旦跑起来，修改的代价极高 . . . 36
10.5 本章小结 . . . 37

11 事件溯源：让事实成为一等公民 37
11.1 三种等价视图 . . . 37
11.2 用事件序列换取解释力 . . . 38
11.3 代价与边界 . . . 39
11.4 术语不必先学，原理必须先懂 . . . 40
11.5 本章小结 . . . 40

12 总结与延伸 40
12.1 讲者留下的结论 . . . 40
12.2 本讲义的压缩 . . . 41
12.3 边界与下一步 . . . 41
12.4 拓展阅读（幻灯片标注的出处） . . . 41

1　引子：写代码变便宜之后，什么变贵了

上一讲把软件工程的历史铺开：从软件危机说起，到 Royce 那张著名的“瀑布”图，再到 traceability 与 UML 这类试图对齐整个世界的工程手段。这一讲要回答一个当下更尖锐的问题：既然 AI 已经能又快又便宜地写出“目标明确”的代码，那么软件工程剩下的事情是什么？本讲给出的答案是**需求**、**产品设计**与**架构**——尤其是第三条“架构”，它平时最容易被忽略，却对学生自己写的软件和真正的业务系统同样致命。

1.1　上一讲留下的三个关键词

课程开场用一页回顾把上一讲压缩成三个关键词：Intent–Spec–Impl Gap、瀑布模型、Data-Dependency Graph。三者说的是同一件事的不同侧面。需求分析、系统设计、代码实现各有自己的表达方式与工具，可它们之间必须反复对齐；Royce 用“瀑布”命名这种自上而下的过程，同时批判了它：如果实现阶段才发现前面某一步错了，错误会沿着数据依赖图往回污染，导致大片工作失效。软件工程的大部分方法，都是在为这张依赖图寻找可控的工程化手段。

traceability：给依赖图装上编号

把需求、设计、测试用例乃至代码片段都赋予稳定标识，让“改一处会影响谁”可以被查询，而不是靠记忆和口头交接。幻灯片把这层意思概括为“UML 试图‘对齐’全世界”：这类建模语言的目标，是为人与人的对齐提供一种结构化、有语义的载体。讲者在复盘时强调：这类手段的价值在于压缩沟通成本，而不在于图画得漂不漂亮。



图 1: 上一讲回顾页：Intent–Spec–Impl Gap、瀑布模型与 Data-Dependency Graph 三个关键词[1]

[1] 视频画面时间区间：00:03:00–00:03:30。若为裁剪图，时间区间仍对应原视频画面。

1.2 “软件不软”：逻辑制品容不下细微差错

课程对软件工程困难的第二个观察来自材料本身的性质。椅子靠背是“软”的：被压瘪之后能弹回来，下一次使用仍然完好。软件完全不是这样。它是纯粹的逻辑制品，具有形式语义，容不下任何细微差错。讲者用两个例子说明这一点。第一个是宇宙射线：如果一段程序运行在卫星上，读入的 `unsigned int` 最高位被粒子翻转成 1，一个本来等于 1 的循环次数会立刻变成一个巨大的数，程序的表现随即失控——它会无穷无尽地读输入，或者在一千次读入之后以内存错误告终。第二个例子更贴近数据中心：随着制程推进、指令密度提高，热噪声可能让同一条指令在不同机器上给出不同结果，这种错误甚至要在服务器运行一段时间之后才暴露出来。

“不是环境问题，是软件问题”

讲者把这种误差称为 silent error：没有崩溃、没有告警，结果却不同。它的存在说明“软件”只是相对硬件而言可改，并不是在语义上可以容忍噪声。把任何无法解释的线上异常都归给环境，等于放弃了软件工程里最基本的严谨性。



图 2: “软件工程”的基本观察：需求难对齐、方案难选择、实现与返工贵[2]

1.3 需求、产品、架构：写代码之前的三件事

同一页幻灯片把困难拆成三条：客户只会自然语言，需求因此难对齐；满足条件的方案很多、各有优劣，因此方案难选择；生产代码很贵、极容易出错，做错之后返工更贵。这条线索解释了为什么课程的学习路径是自下而上的：先写得出代码、体会过开发的痛苦，才可能理解成本与边界，才可能做出靠谱的需求与产品判断。

而生成式 AI 改变了代价结构。幻灯片把当下的局面写成“Agentic AI：软件生产力的杠杆”——**生产目标明确的代码，极为便宜**，模型还能并行铺开 agent swarm，也能写测试用例、断言甚至证

[2]视频画面时间区间：00:05:30–00:06:00。若为裁剪图，时间区间仍对应原视频画面。

明。于是剩下的杠杆只剩三根：需求、产品设计、架构。讲者的实践印证了这一点：他现在让 AI 生成代码时“真的已经不看它的实现了”，只要把边界划定清楚、并且能给出一个像样的评估方式，就可以放心把工作铺开。

技术债（technical debt）：每个决定都在借钱

在做 MVP 的时候做的任何决定，都等于借了一笔债：因为那一刻你并不知道这个决定好还是不好。讲者举的例子是 MVP 的功能选择——按钮样式依赖界面设计，界面设计依赖功能实现，在最不确定的时候去调按钮，很可能随后被整体推翻。反过来说，能识别出“哪些功能最重要”，正是 MVP 的价值；而随手写进项目的框架、数据库与目录结构，都会以历史的形式留在 agent 的上下文里，成为后续每一次改动都要背负的债务。

幻灯片里还有一层提醒：AI 让做 MVP 前所未有的容易，也**制造了一种“驾驭了 AI”的错觉**。AI 确实可以沿着既有代码一路往后改，迭代三个版本都不成问题；可是当系统在真实环境里跑了足够久，当年的设计假设不再成立时，那些“暗角”就会露出獠牙。

Wicked Problems：连“什么问题”都没有共识

幻灯片引用城市规划领域的概念来解释需求研究之难：不同参与者可能连“什么算成功”都没有共识（Rittel & Webber, 1973）。因此，“需求、产品、架构太难定量研究”不是学术界偷懒，而是问题本身的性质；这也解释了为什么学校里的软件工程研究者大多集中在与编码、实现相关的题目上。



图 3: 软件工程方法与研究：需求、产品、架构难以定量研究，以及城市规划的 Wicked Problems 参照[3]

[3]视频画面时间区间：00:08:30–00:09:00。若为裁剪图，时间区间仍对应原视频画面。



图 4: Agentic AI 的杠杆：当“目标明确的代码”变便宜，需求、产品设计、架构成为撬动生产力的支点[4]

1.4 一段事故插曲：可观测性也是债

讲到第 14 分钟时发生了一次意外：他的键盘鼠标全部失效，屏幕上的系统 UI 仍在正常工作，OBS 也还在推流，但他自己无法操作任何设备——这一段的视频与音频也因此不完整。讲者随后把这件事当成软件工程案例来分析：出了问题要去看日志，这就要求写软件时输出足够多的日志；而如果你不明确要求，AI 不会主动替系统补上日志与可观测性，它会交出一个“没有任何日志”的系统，这笔技术债照样要还。

1.5 本章小结

软件工程之所以困难，一方面来自需求的不可对齐，另一方面来自软件作为逻辑制品的零容错。生成式 AI 把“生产目标明确的代码”的成本压到极低，于是人的注意力应当转移到三根杠杆上：需求、产品设计、架构。它们决定了一个软件系统能走多远，也决定了技术债以什么速度累积。接下来的两章从需求开始，用讲者最喜欢的一个反面教材——南京大学教务系统——来说明需求为什么会变成一场“史诗级的需求爆炸”。

2 需求工程：从一个真实的教务系统说起

2.1 一个“史诗级需求爆炸”的样本

需求工程在课堂上很难讲，原因是学生没有真正的客户：你不会去见教务处的人，也不敢拍胸脯用 20 万元承包学校的教务系统，更不敢承诺“交付时达不到功能要求怎么办”。但 AI 时代让这件事有了新的入口：独立开发者数量爆炸，付费用户则第一次面临真正的竞争。

[4]视频画面时间区间：00:11:30–00:12:00。若为裁剪图，时间区间仍对应原视频画面。

于是讲者直接打开教务系统现场演示。这个系统被他称为“史诗级的需求爆炸”，因为它几乎是一个业务系统演化史的标本：从学生端到教师端，从课表到教学任务，从教材选用到大纲查看，每一项功能都能追溯到某一次具体的行政需求。

2.2 需求是叠加出来的

演示里最值得记录的不是某个按钮，而是功能的**叠加痕迹**。教师端可以查看我的教学任务，其中又分当前教学学期、最新排课学期与历史授课信息；期末签到表只在期末考试期间允许点击，没到时间会弹窗提示无法打印；编辑教材会跳到另一个页面；查看大纲刷新很慢；助教管理里既有“课表内教学班”的助教，又有一类“慕课助教”。讲者当场指出：慕课这件事在疫情之前根本不存在，疫情催生了慕课功能，慕课又催生了“慕课助教”这个独立角色，而删除与新增还要遵守“请在可维护时段内操作”的窗口限制。

需求爆炸：不是设计出来的，是长出来的

一个运行了二十年的业务系统里，绝大多数复杂度都来自“后来加上的那一条”。单看每一条需求都合理，合起来就变成一个没人能整体理解的庞然大物。这解释了讲者后面为什么会说：面对这样的系统，第一步就应该考虑架构，而不是先堆一个能跑的原型。

表 1: 演示中出现的一小部分教师端功能及其可追溯的叠加来源（据视频画面与讲解整理）

功能	叠加痕迹
我的教学任务	需要同时容纳“当前教学学期”“最新排课学期”“历史授课信息”三种口径
期末签到表	只在期末考试期间开放，未到时间要给出明确的失败提示
教材选用与大纲	分属不同页面，编辑教材另有弹窗；大纲页面刷新慢，体验被视频点名批评
助教管理	从“课表内教学班助教”扩展出“慕课助教”，形成史诗级 bug
新增/删除操作	受“可维护时段”限制，因为系统要对着真实的学生与教师数据运行
智能问答	系统里悄悄加上了 AI 入口，可以把“仙林教学楼的灯归谁管”这种问题转到基建处
教学周历	讲者曾现场演示过直接用 AI 把它填满，事后没有任何人审计填报内容

讲者顺带做了一次成本比较：这样一个花了不少钱的系统，其价值在某种程度上不如把预算换成 token 补贴给师生——他自己用 agent 去查一次问题，成本大约只要 0.1 0.2 元。

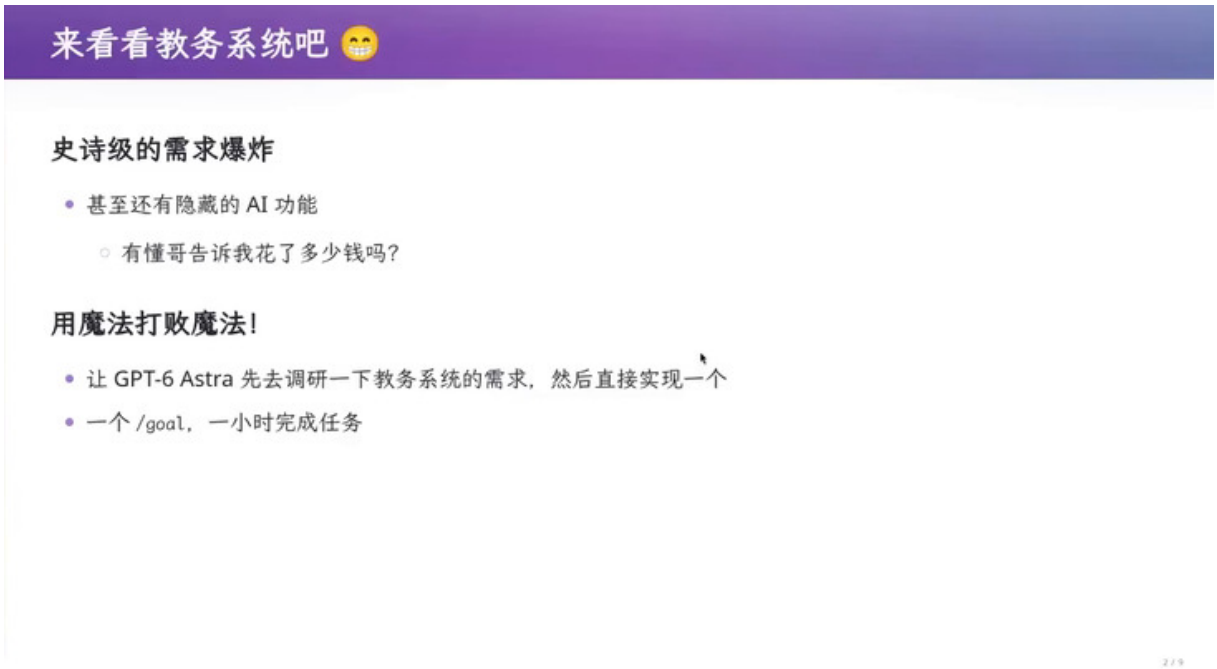


图 5: “来看看教务系统吧”：史诗级的需求爆炸与隐藏的 AI 功能[5]

2.3 本章小结

教务系统说明了需求工程的第一层现实：需求不是一次性写下来的规格，而是历史层层叠加的结果。理解这一点，才能理解为什么“把需求方访谈一遍”这种经典做法在真实系统上会失效。下一节换一个方向提问：如果让一个足够强的模型来做需求调研，它交出来的东西能不能用？

3 把需求工程交给模型：一份真实的需求说明书

3.1 “用魔法打败魔法”：先调研，再实现

讲者的做法是让模型先去调研教务系统的需求。幻灯片上写着“用魔法打败魔法！让 GPT-6 Astra 先去调研一下教务系统的需求，然后直接实现一个”。调研的输入只有教师视角的真实页面观察，产出是一份《南京大学本科教学服务平台需求说明书》，副标题注明“教师视角 · 软件工程课程案例”“版本：1.0（调研与候选需求基线）”“调研日期：2026 年 9 月 21 日”。讲者评价这份文档“很不错”，并强调它是一份**真实**的需求报告案例：课堂上通常只能看到企业中截取的一小段示例，而这一次的输入是学校里每天都在用的系统。

3.2 需求说明书写了什么

文档目录本身就有教学价值：调研方法、证据与边界；业务目标与成功标准；系统范围、参与者与边界；教师工作旅程与功能地图；角色与数据授权模型；功能需求目录；详细用例规约；最后还有术语表、文档与配套文件。讲者指出，这份文档之所以要写这么多，是因为它要面对真正的客户：一个术语在双方理解不一致时可以被查阅，语义就此对齐。

[5] 视频画面时间区间：00:18:00–00:18:30。若为裁剪图，时间区间仍对应原视频画面。

更关键的是它自带 traceability。功能需求目录里，每一条需求都有编号与优先级（例如 FR-001 / M），有“系统应提供的行为”，有“候选验收口径”，还有一列“依据”；依据里用 O 表示该功能或字段来自实际观察，用 P 表示属于拟议的实现约束，并明确写出“未对现网进行越权测试”。业务目标同样有编号（例如 BG-01），可以被需求条目引用。于是这张表不只是需求清单，还是一张可以查询的依赖图。

需求 ID 是给“人”用的，不是给数据库用的

“教学班和学生管理有关”这种自然语言句子无法被机器追踪；一旦需求获得稳定编号与形式语义字段，就能回答“改这条需求会影响哪些验收场景”“哪个团队负责哪几条”。幻灯片把这层价值总结得很直白：有了 traceability，一个管理者就能把团队管起来，前二十条需求交给这个团队、后二十条交给那个团队，进度落后一眼可见。

表 2: 需求说明书的骨架与它对齐的对象（据视频画面整理）

文档部分	对齐的对象
调研方法、证据与边界	说明“哪些结论来自观察、哪些只是拟议”，避免把推测当事实
业务目标与成功标准	让不同角色先对“什么算成功”达成共识
系统范围、参与者与边界	划清谁能用、谁不能用，避免无边界扩张
教师工作旅程与功能地图	用任务流程而非功能列表组织需求
角色与数据授权模型	明确数据可见性与身份切换规则
功能需求目录	ID、优先级、行为、候选验收口径、依据（O/P）
详细用例规约与术语表	把异常场景与专有名词固定下来，供人与模型共同引用

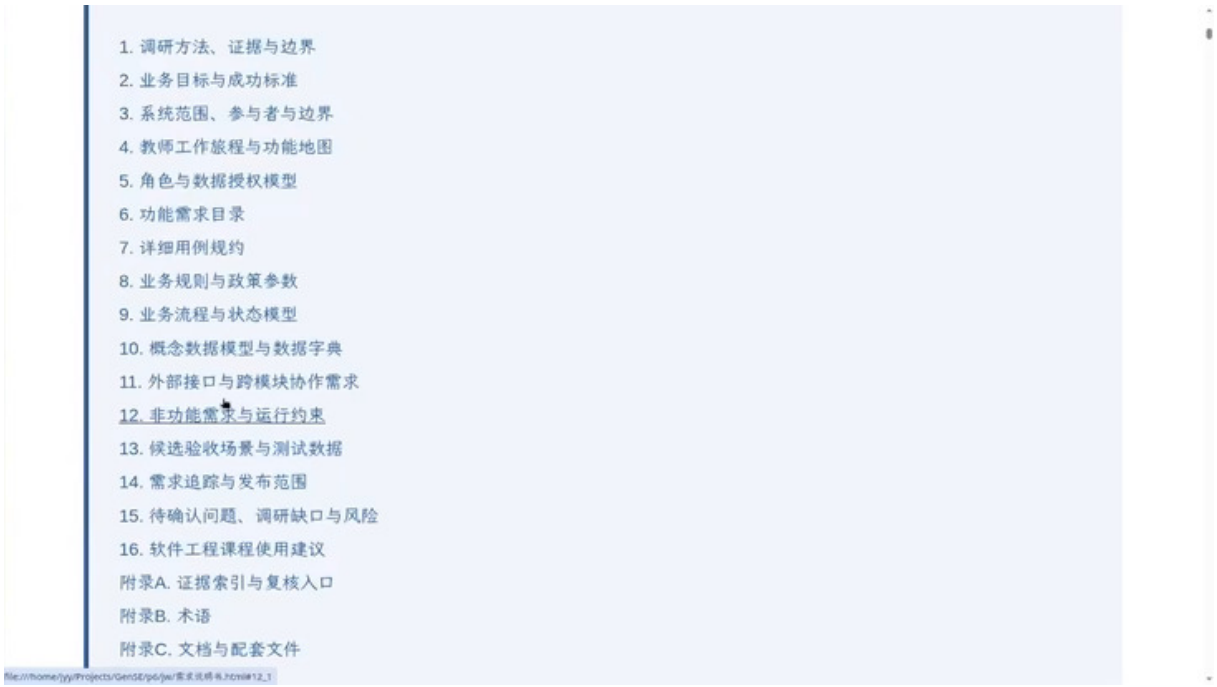


图 6: AI 生成的需求说明书目录：从调研方法与证据边界，到功能需求目录与配套文件[6]


表中"依据"包含O/P与证据编号。O表示功能或字段来源于观察，右侧验收口径仍是待执行的候选测试；涉及未观察的服务器约束时按P处理。每个模块的详细步骤与异常场景见第7章。

6.1 门户与身份

ID / 优先级	系统应提供的行为	候选验收口径	依据
FR-001 / M	提供本科教学服务分类、服务检索和应用入口。	教师可定位六类本科服务，并从入口到达对应应用。	O · E01
FR-002 / M	当一个服务支持多种已分配身份时，提供身份选择并显示当前身份。	选择本科教师组后页面显示该角色；未分配角色不得成为可选项。	O · E12/E13
FR-003 / M	服务页面明确当前学期、学年、届次或业务批次，允许在授权范围切换。	改变上下文后列表同步更新；无记录时保留所选上下文。	O · E02/E08/E09/E21
FR-004 / M	在门户与子应用之间复用统一身份认证，并妥善处理认证失效。	跨应用免重复登录；会话失效时说明原因，未提交内容按约定恢复且不得误提交。	P · E01/E12

6.2 教学任务

ID / 优先级	系统应提供的行为	候选验收口径	依据
FR-005 / M	分别展示当前教学学期、最新排课学期和历史	三个入口可区分；历史任务按指定学期及课程条件	O · E02

图 7: 功能需求目录 6.1 门户与身份：ID/优先级、系统应提供的行为、候选验收口径与依据[7]

3.3 只读需求说明书，能做出一个系统吗

紧接着讲者做了第二次实验：启动一个 subagent，要求它**不看任何其他资料**，只依据这份需求说明书实现一个“假”的教务系统。结果让他重新有了信心——生成的系统在功能层面已经和真实

[6]视频画面时间区间：00:24:30–00:25:00。若为裁剪图，时间区间仍对应原视频画面。
[7]视频画面时间区间：00:26:00–00:26:30。若为裁剪图，时间区间仍对应原视频画面。

系统很像：可以查看教学任务与教材管理，可以保存记录，也能看到学生名单。这带来一个直接结论：当需求文档写到那个程度时，编码、追踪、测试这些环节，AI 基本都能接手。

但演示也暴露了边界。讲者试着录入一次不合理的缺课记录，发现系统没有拦住他——因为需求说明书里并没有定义这条业务规则的上界。这份文档能保证“不遗漏讲过的东西”，却无法凭空发明“没被想到的约束”。

文档完备不等于需求确定

需求说明书是对齐沟通的工具，不是需求的终结。它把已知的观察整理清楚，但真实系统里那些“暂时没人想到、将来一定会出现”的情况仍然存在。把文档完备误当成需求确定，是接下来一切痛苦的起点。

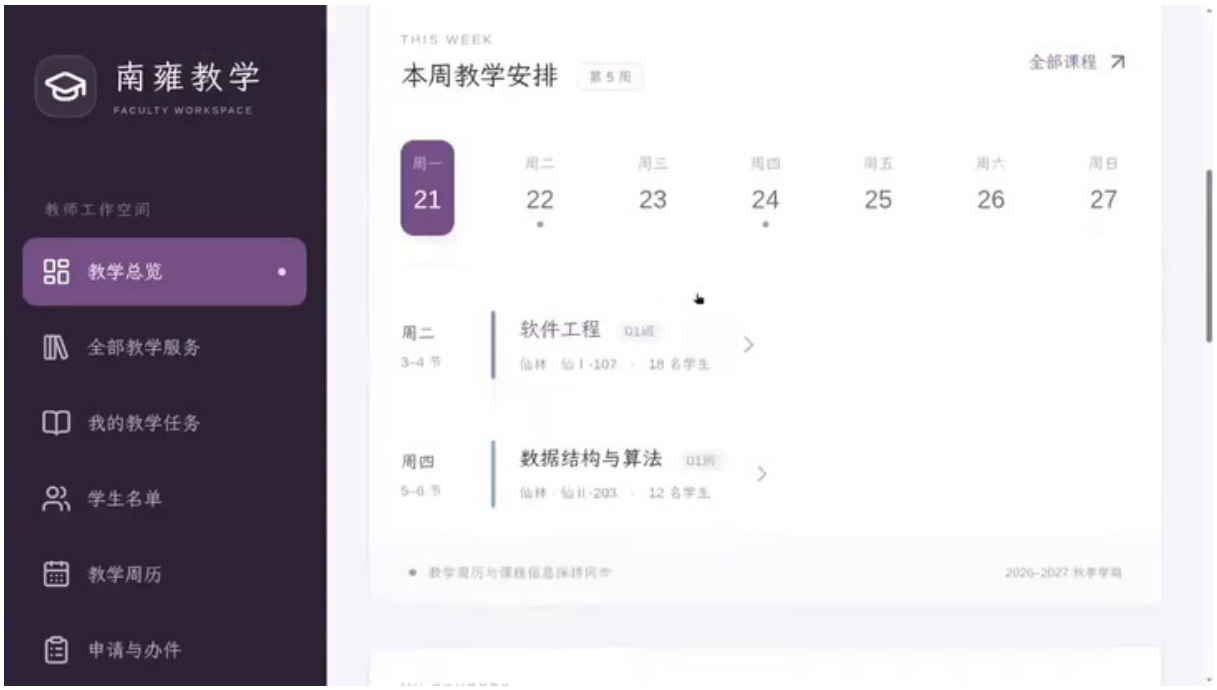


图 8: 只依据需求说明书生成的假教务系统：界面与真实系统已经相当接近[8]

3.4 本章小结

大模型可以承担需求工程的体力活：调研、整理、编号、写成清单，甚至按说明书实现一个原型。这让“需求只有一份文档就能开工”第一次变得可行。但文档解决的是沟通与追踪，不解决需求的本质不确定性。下一章从“人”和“变化”两个方向，说明这种不确定性为什么无法被消灭。

4 需求的另一半：人、领域知识与不确定性

4.1 为谁设计：同理心与用户画像

讲者在演示教师端时反复提醒一件事：教务系统的用户不是你自己。真实的使用者中，有本科学历、53 岁、还有两年就要退休的教务员，她上大学时可能都没有操作过电脑。对这样的用户来说，

[8] 视频画面时间区间：00:30:30–00:31:00。若为裁剪图，时间区间仍对应原视频画面。

需求的重点不是“最好的数据库、最高的性能”，而是字大一点、点鼠标时不要手滑到旁边的按钮上。幻灯片用“要为教务员设计，也要为年轻人设计”来概括这种多目标设计，并把这种能力称为共情。

用户画像：把“用户”从抽象名词还原成人

“教务员”这个抽象名词给出的信息量几乎为零。加上年龄、学历、操作习惯与工作压力，才能判断一个界面该不该有下拉框、要不要给出明确的失败提示、按钮间距是否足够。讲者把这类软技能看成 CS 同学最容易被忽视的部分：它是需求阶段最容易拉开差距的能力。

4.2 领域知识与行业壁垒

需求难做还有一层原因：不同人对业务的掌握程度不同。讲者举了两个方向。第一，如果做需求的人有行业经验——比如软件公司已经在这个行业摸爬滚打二十年，他知道教务员想要什么、知道学位授予的流程该怎样变成需求清单，那么他设计出的东西偏差就更小；而一个刚毕业的团队只能从零开始，被迫重新走一遍别人的经验。第二，需求本身会变，而且是**在系统已经运行之后**变。讲者把这句话说得很直接：需求永远不可能说清楚，而且需求不是一成不变的。

4.3 案例：培养方案、学分与“最后的暗箱操作”

幻灯片给了一个“虚构但可能发生”的案例：2024 级李同学在 2026 年转入计算机专业，适用目标专业的 2024 版方案。毕业至少 160 学分：通识 40、专业必修 72、专业选修 28、实践 20，各类分别达标；数学必修组二选一，高数 A（5 学分）或高数 B（4 学分）加桥接课（1 学分）；算法课须先修数据结构，缓考生可暂选，开课前仍未通过则撤销；两门交换课程获批合抵算法课，认定 4 学分，且不能再计入专业选修；重修学分只计一次，成绩单保留历次记录、绩点按较高成绩计算。

这些规则可以写成一条式子：

$$160 = 40_{\text{通识}} + 72_{\text{专业必修}} + 28_{\text{专业选修}} + 20_{\text{实践}}$$

其中 160 是毕业所需的最低总学分，右侧四项分别是四个课程类别的学分要求，“各类分别达标”意味着总和相等并不足以判定毕业。真正的问题在规则之外：每一条规则在被写进系统的时刻都是合理的，可一旦教学计划调整，这些规则就开始互相冲突。

讲者因此讲了他很在意的一个更激烈的版本：学校把离散数学这门课拆成上下两学期两门课，准出结果和保研要求随之变化。原来的系统里，毕业要求是“选修了离散数学”；现在必须允许“选修了离散数学一，或者选修了离散数学二”。于是首页上那个“毕业要求”界面要加一个“或”的关系，下拉框里要能表达这种关系——每一次这样的叠加，都在给史山加一层。更麻烦的是抵扣：软院也有一门离散数学，转院系的学生用它能不能抵扣计算机学院的离散数学一和二，还是只能抵消其中一门？面对开放的物理世界，这类情形无穷无尽。

而最终，系统里一定会留下一个“最后的暗箱操作”。幻灯片写得很坦白：管理员凭线下审批，点“视为满足”覆盖指定学生的未达标项，并记录理由、批准人和适用方案。讲者补充了现实版本：一个已经拖到第六年、只因一门课拿不到学位的学生，教学主任认为新旧两门课之间存在替代关系，于是打电话给工程师，让人登录数据库把这位同学的毕业条件改成满足——好在审批链条还在，档案可查，在人类世界的意义上这是一个有解释的例外。

例外不是 bug，而是在给“无法建模的世界”开洞

把例外写成“点一个按钮覆盖”，看起来解决了当下的问题，代价是系统里出现了无法用规则解释的数据。这类数据的数量会随时间增长，最终变成系统里的黑洞：你知道某个结论存在，却说不清它是怎么来的。

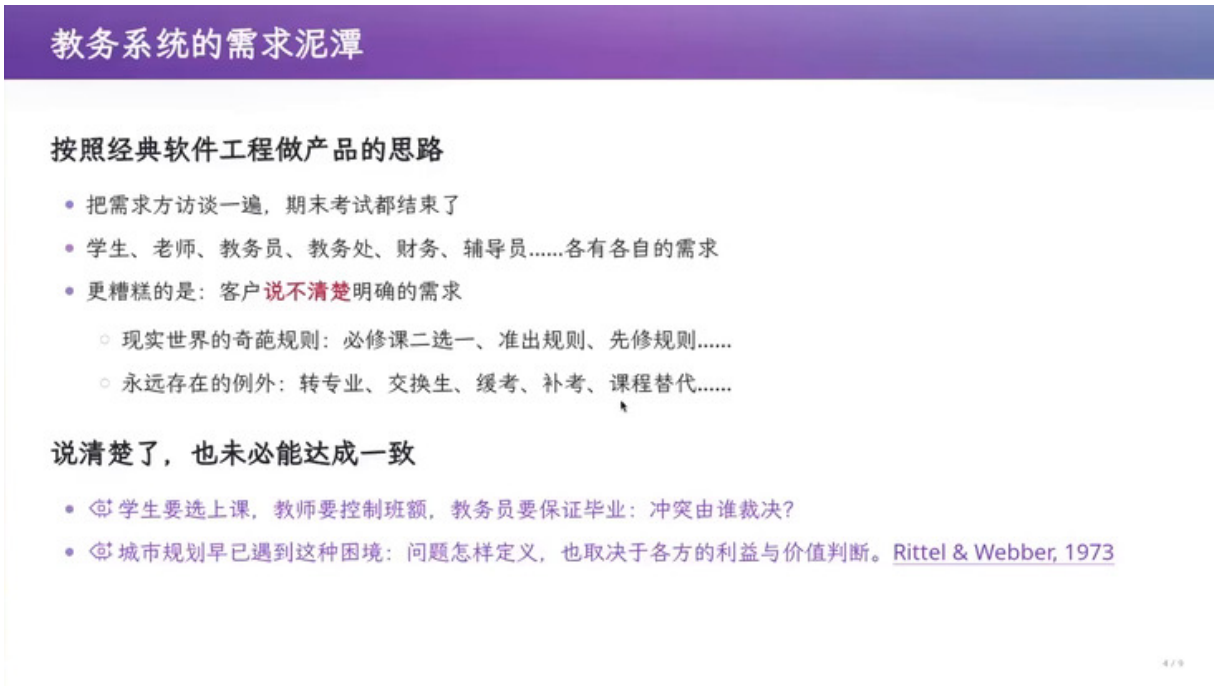


图 9: 教务系统的需求泥潭：多角色冲突、说不清的需求，以及现实世界的奇葩规则[9]

[9]视频画面时间区间：00:35:00–00:35:30。若为裁剪图，时间区间仍对应原视频画面。

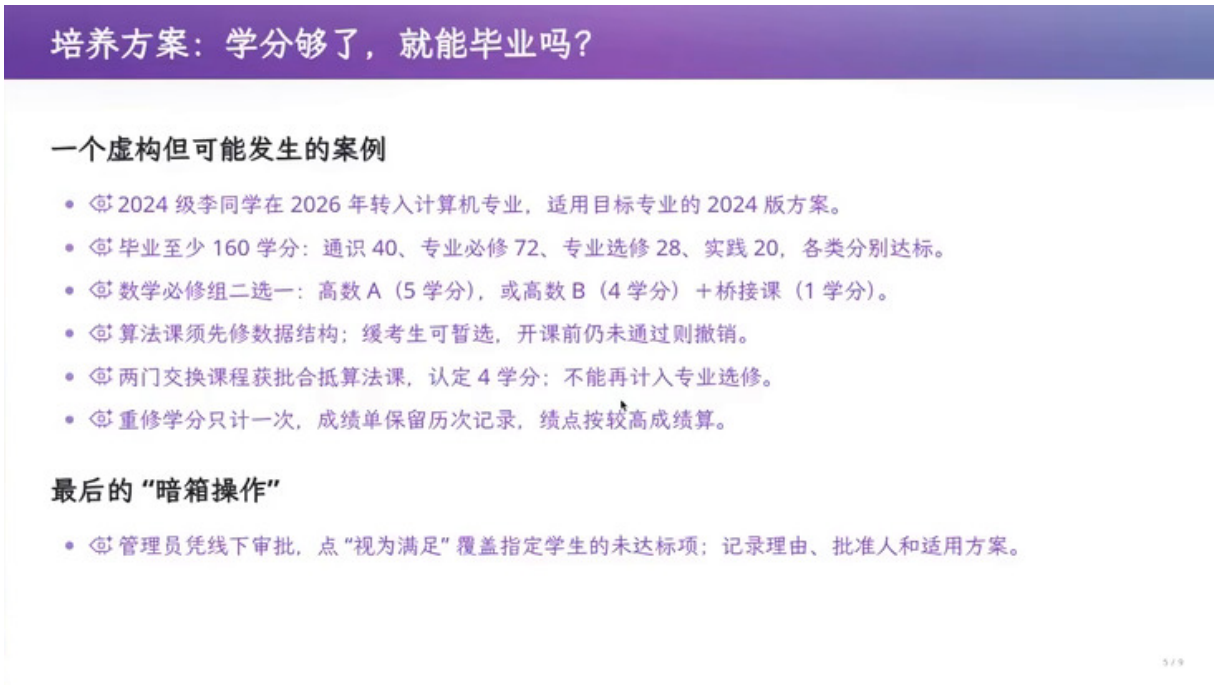


图 10: 培养方案案例：学分结构与各类规则要求“分别达标”，最后还留了暗箱操作[10]

4.4 一次亲手制造的事故：把缺课次数改成 -1

需求被“勉强实现”之后，系统就进入了讲者所说的焦油坑。他分享了自己 2018 年第一次用教师端教务系统的经历：那套系统允许教师设置学生的缺课次数，他绕过前端的输入校验，把一位同学的缺课次数从 0 改成了 -1。前端确实做了校验——直接输入一个极大的数字会被拒绝——但 -1 从后端进去了，随后所有能选出这位同学选课记录的页面全部瘫痪，直接返回 500。最终的处理方式也是现实的：打电话给教务，工程师第二天登录数据库把那条记录修掉。

[10] 视频画面时间区间：00:37:30–00:38:00。若为裁剪图，时间区间仍对应原视频画面。



图 11: “勉强实现需求，然后进入焦油坑”：把缺课次数改成 −1 的现场记录[11]

4.5 没有银弹

这些经验指向一个经典结论。幻灯片上写着“银弹：传说中的‘一招制胜’”：西方传说里普通子弹奈何不了狼人，银弹却能把它制服，Brooks 用它比喻人们期待的“软件工程万能药”（Brooks, 1987）。两年后的结论依旧成立：复杂需求与持续变更没有一招制胜的解法。需求的变更会穿透整个软件工程的依赖图——“允许两门交换课程合抵一门必修课”这一条规则，会同时牵动课程对应、学分统计、毕业审核、成绩单与测试用例。新规则上线之后，已认定的学分与已批准的特例又成了新需求的约束。

为什么“重写一遍”不是银弹

在一个课程作业里，你可以随时把项目清零、从头再做。但一个已经上线的系统会持续与物理世界交互，它必须生存、必须挣钱，于是只能背着技术债维护下去。讲者的判断是：如果前面做得不够前瞻，维护成本会随着软件增长不断恶化。

[11] 视频画面时间区间：00:42:00–00:42:30。若为裁剪图，时间区间仍对应原视频画面。



图 12: 银弹传说：复杂需求与持续变更没有万能解法（幻灯片标注 Brooks, 1987）[12]

4.6 本章小结

需求的困难有三个来源：用户的多样性（需要共情与画像）、领域知识的门槛，以及需求本身的不确定性——说不清、且会变。它们无法靠更勤奋的调研消除，只能靠架构去吸收。于是问题变成：什么样的架构，能让一个软件在需求持续膨胀与改变时仍然保持韧性？

5 架构：控制复杂性的手艺

5.1 架构不是玄学

谈到架构，人们容易想到框图、数据库、微服务、流式数据处理这些名词，仿佛它是资深工程师的特权。讲者的判断正相反：架构从你写下第一行代码时就存在，本质只有一件事——**控制复杂性**。哪怕只有十行代码，也已经有了架构：写一个 `main`，依次调用读入、解析、处理、输出四个函数，就是在决定“复杂度放在哪里”、“变化会波及到哪里”。幻灯片把这一节的标题写成“软件架构：真正决定软件品味和能走多远的决定性因素”，并把作用归纳成两条：驾驭问题的复杂性（简化问题、安放复杂性、减少依赖），以及应对需求的变化（降低维护成本）。

架构＝把变化安放在边界之内

幻灯片引用了一种更精确的表述：把容易变化的设计决策藏在模块内部，让一次变化尽量只影响一个地方（Parnas, 1972）。这句话给出了判断架构好坏的操作性标准：不是“看起来是否高大上”，而是“一次改动会沿着依赖图走多远”。

[12] 视频画面时间区间：00:43:30–00:44:00。若为裁剪图，时间区间仍对应原视频画面。

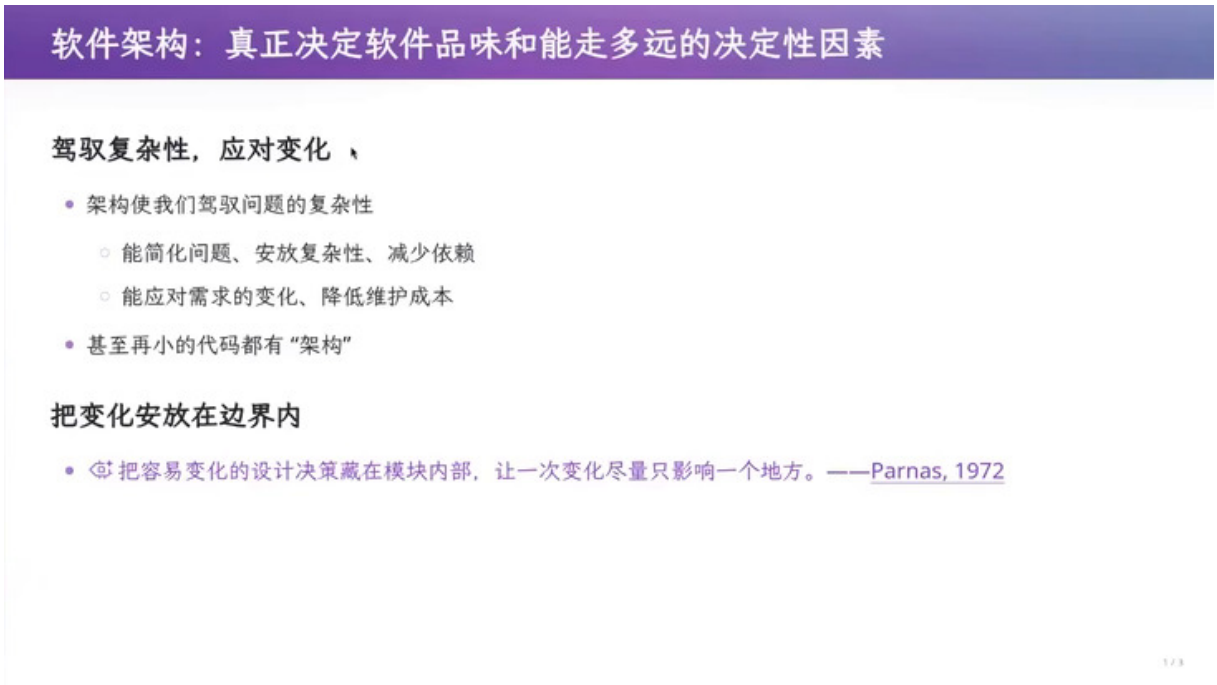


图 13: 软件架构：驾驭复杂性、应对变化；把容易变化的设计决策藏在模块内部[13]



图 14: 再小的代码也有架构：四个函数构成的状态机，以及“语言与架构”的关系[14]

5.2 最小的一段代码也有架构

幻灯片给出的最小例子是四个函数组成的流水线：

[13]视频画面时间区间：00:45:30–00:46:00。若为裁剪图，时间区间仍对应原视频画面。
[14]视频画面时间区间：00:46:00–00:46:30。若为裁剪图，时间区间仍对应原视频画面。

```c
int main(int argc, char *argv[]) {
    read_input();
    parse_input();
    process_input();
    write_output();
}
```

Listing 1: 四函数流水线：最小但仍然真实存在的架构

它看起来平淡，却回答了架构问题：数据怎样穿过各阶段？换一种输入格式，需要改到哪里？讲者由此批评“老登式的语言律师教学”：如果讲 C++ 只讲“lambda 就是 `auto f = [](int a, int b){ return a + b; }`”，那么讲了和没讲没有区别。语义的价值在于把设计意图变成约束——谁能改数据、谁负责资源、谁可以依赖谁。类似的联系在库函数里随处可见：命令行程序处理 `argv` 时如果只想到 `string compare`，就会写出难以维护的分支；而世界上早有一个东西叫 `getopt`。

5.3 `getopt`：把复杂性隔离成一个解析状态

讲者用 `getopt(3)` 的手册页说明什么叫“把复杂性隔开”。命令行参数本身极其复杂：短参数、长参数、Unix 风格与各种约定；在 C 的时代，手写一个完整的解析器非常费劲。POSIX 给出一组约定之后，`getopt`、`getopt_long` 把它封装起来，使用者的代码就只剩一个 `while` 循环加一个 `switch`：先得到解析状态，再从状态出发决定下一步做什么。

十行的架构：解析状态就是架构

`getopt` 之所以算架构，是因为它把“参数千变万化”这件事收敛到一个中间表示上。后续逻辑不再直接面对原始字符串，而是面对结构化的解析结果。讲者总结这种模式的通用性：先解析成状态，再往下走——后面在 Excel 二维码里，同样的思路会以“中间语言”的形式再出现一次。

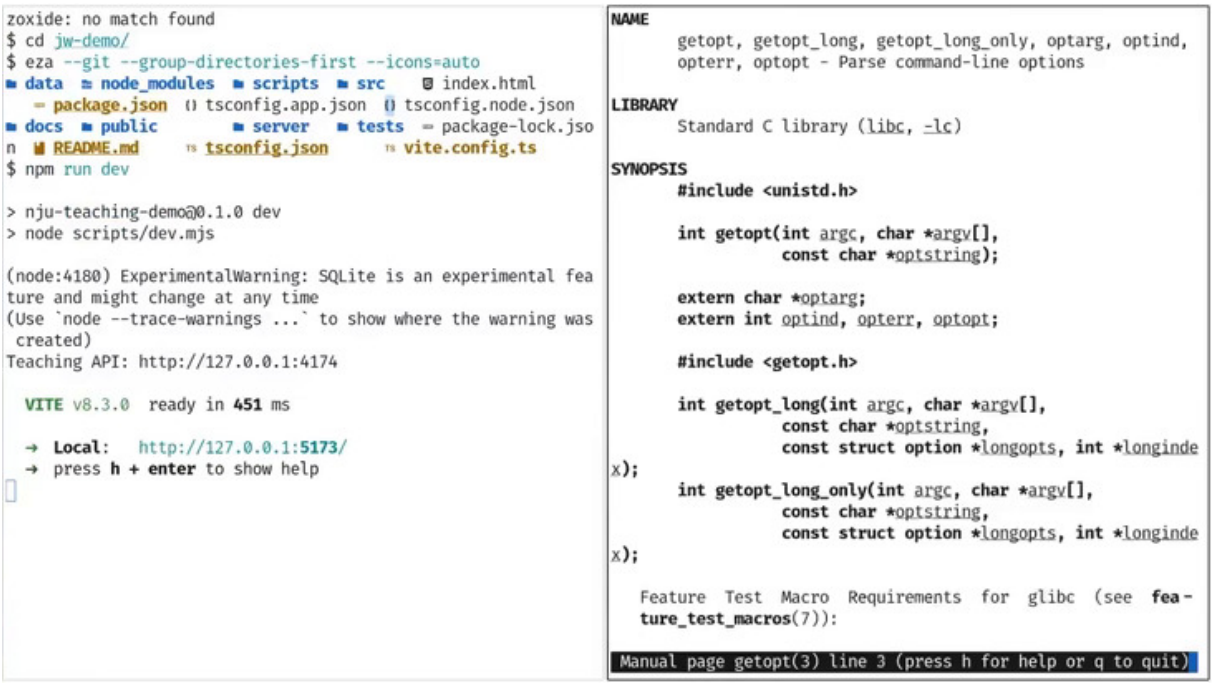


图 15: `getopt(3)` 手册页：`argv` 的复杂性被封在 POSIX 约定的解析状态里[15]

5.4 本章小结

架构不是大项目才需要的装饰，而是每一段代码都在做的取舍：把复杂性放在哪里、让变化影响多远。`getopt` 提供了最小尺度的范例——用一层中间状态把混乱隔开。下一章把这个标准推到极端：同一段只有十行的算法，用四种不同的表示写出来，架构水平的高低会暴露得非常清楚。

6 十行的架构：非递归汉诺塔的四种表达

6.1 为什么大家都写不出来

讲者喜欢用非递归的汉诺塔作为最小的架构样本：他曾经问刚入学的研究生能不能写出非递归版本，得到的答案是摇头。幻灯片上的问题是“大家到底为什么写不出来”，答案不在于难度，而在于**对递归如何被机器执行理解得不够深**。而同一个问题换一种表示，难度就可能改变——幻灯片引用了一项同构变体实验作为旁证（Zhang & Norman, 1994）。于是这一节变成了一场“同一道题，四种表达”的对照实验。

6.2 第一种：GPT-6 的数学规律

第一个版本来自模型。幻灯片直接把它的态度写成标题：“GPT-6：反正我不做人类了，你写不出来是因为你数学不如我好。”它没有模拟递归，而是找到了汉诺塔的数学性质：**盘号等于步号二进制末尾 0 的个数加一，并且每个盘子的移动方向固定**。用公式写出来就是

$$\mathrm{disk}(k) = 1 + z_2(k)$$

[15] 视频画面时间区间：00:47:30–00:48:00。若为裁剪图，时间区间仍对应原视频画面。

其中 k 是第几步（从 1 开始计数），$\mathrm{disk}(k)$ 是第 k 步应当移动的盘子编号，$z_2(k)$ 是 k 的二进制表示末尾连续 0 的个数。末尾 0 的个数越多，说明该盘在这一步越“靠上层”：例如第 4 步（二进制 100）末尾有两个 0，于是这一步移动第 3 号盘。代码把这条规律直接翻译成循环：

```
for (uint64_t step = 1; step <= total; step++) {
    int disk = 1;
    for (uint64_t x = step; (x & 1) == 0; x >>= 1) disk++;
    int dir = (n - disk) % 2 == 0 ? -1 : 1;
    int from = position[disk];
    int to = (from + dir + 3) % 3;
    printf("Move␣disk␣%d:␣%c␣->␣%c\n", disk, pegs[from], pegs[to]);
    position[disk] = to;
}
```

Listing 2: GPT-6 的非递归汉诺塔：用数学规律替代模拟

讲者对这段代码的评价很有代表性：他看不懂，但相信它是对的，而且“highly optimized”。这类解的全部复杂度都在数学洞察上——一旦知道规律，实现几乎是平凡的。

6.3 第二种：DeepSeek 的任务栈

第二个版本走了教科书路线。幻灯片写成“DeepSeek：人类语料里似乎让我这么干”，它实现的是大家熟悉的那句话——递归就是栈：把尚未完成的工作显式地压进任务栈，按 LIFO 反过来压栈，于是总能先执行栈顶的最小子任务。

```
push(&s, n, A, B, C);
while (!isEmpty(&s)) {
    Task t = pop(&s);
    if (t.n == 1) {
        printf("%c␣->␣%c\n", t.src, t.dst);
    } else {
        push(&s, t.n - 1, t.aux, t.src, t.dst);
        push(&s, 1, t.src, t.aux, t.dst);
        push(&s, t.n - 1, t.src, t.dst, t.aux);
    }
}
```

Listing 3: DeepSeek 的非递归汉诺塔：把递归展开成待办事项

讲者称它“非常干净”，因为复杂性被放在了一个显式的数据结构里：任务栈。至于第三种尝试，讲者试了豆包，方案选择与 DeepSeek 相同，但实现有瑕疵，因此没有展开；而当他到搜索引擎上找“非递归汉诺塔”的第一篇博客时，看到的是比 AI slop 更糟的代码——连 `std::move` 都被误用，评论区却一片“干货满满”。

6.4 第三种：把 C 语言的小步语义写成状态机

讲者自己的版本来自另一个方向。他不把汉诺塔看成算法，而把它看成一段需要被“执行”的程序：C 语言有形式语义，一次 single step 对应一个**小步操作语义**（operational semantics），例如“移动 $n-1$ 个盘子”在语言世界里就是一个定义良好的一步。于是可以写一个 C 语言解释器来模拟它，把函数调用显式化成帧栈：调用就压入新帧，返回就弹出旧帧，帧里保存 `pc`（下一条要执行的语句）与参数。

```c
#define call(...) (*(++top) = (Frame){ .pc = 0, __VA_ARGS__ })
#define ret() (--top)

call(n, from, to, aux);
while (top >= stk) {
    Frame *f = top;
    int next_pc = f->pc + 1;
    switch (f->pc) {
    case 0: if (f->n == 1) { printf("%c␣->␣%c\n", f->from, f->to); ret(); } break;
    case 1: call(f->n - 1, f->from, f->aux, f->to); break;
    case 2: call(1, f->from, f->to, f->aux); break;
    case 3: call(f->n - 1, f->aux, f->to, f->from); break;
    case 4: ret(); break;
    }
    f->pc = next_pc;
}
```

Listing 4: “绿导师”的非递归汉诺塔：把函数执行机制显式化

这段代码可以处理任意程序，用在汉诺塔上属于“overkill”；而只要做一步简化，它就能退化成 DeepSeek 那种任务栈版本。讲者的意思是：两个版本等价，从哪个版本出发取决于你想理解什么。

表 3: 同一个汉诺塔的四种表达：复杂性被放在哪里（据幻灯片“同一个汉诺塔，四种表达”整理）

版本	中间表示	复杂性的归宿
GPT-6	步号 → 盘号，`position + step`	把递归树压缩成数学规律，依赖汉诺塔的特殊结构
DeepSeek	任务栈（LIFO，反序压栈）	把递归展开成待办事项，显式管理尚未完成的工作
“绿导师”	帧栈 + `pc`（调用压帧、返回弹帧）	把函数执行机制显式化，理解递归如何被机器执行
搜索结果	分支与细节	代码能否运行之后仍要解释规则；改需求时哪些知识能保留

选择一种表示，就是选择把哪些事情变得容易

幻灯片用一句话总结这场对照实验：选择一种表示，就是选择把哪些事情变得容易。数学规律把“步”作为主变量，任务栈把“未完成的工作”作为主变量，帧栈把“函数执行机制”作为主变量。它们都能跑出正确答案，但面对需求变化时保留的知识不同——这正是架构品味在十行代码尺度上的体现。



图 16: 第一种表达：GPT-6 的数学规律——盘号等于步号二进制末尾 0 的个数加一[16]



图 17: 第二种表达：DeepSeek 的任务栈——把递归展开成待办事项[17]

[16] 视频画面时间区间：00:52:30–00:53:00。若为裁剪图，时间区间仍对应原视频画面。
[17] 视频画面时间区间：00:53:30–00:54:00。若为裁剪图，时间区间仍对应原视频画面。



图 18: 第三种表达：把 C 语言的小步语义显式写成状态机（调用压帧、返回弹帧）[18]

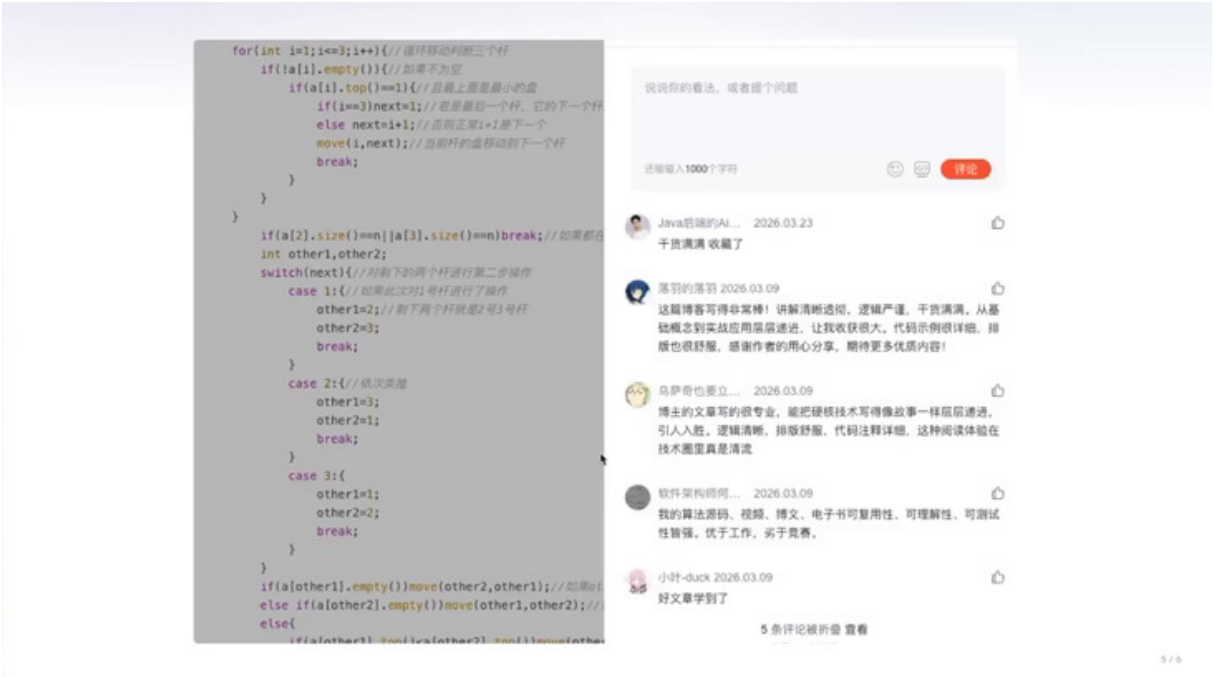


图 19: 搜索排名第一的“非递归汉诺塔”博客：代码分支繁琐，评论区却评价很高[19]

[18] 视频画面时间区间：00:54:30–00:55:00。若为裁剪图，时间区间仍对应原视频画面。

[19] 视频画面时间区间：00:56:00–00:56:30。若为裁剪图，时间区间仍对应原视频画面。

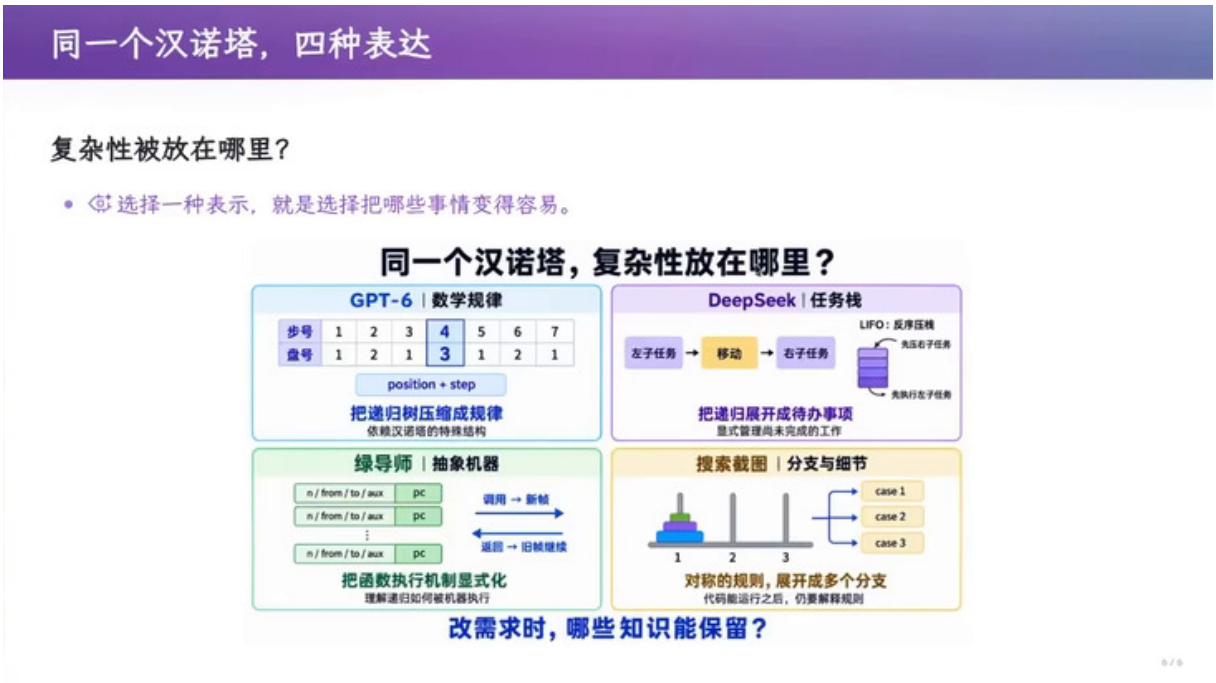


图 20: 四种表达的对照页：复杂性分别被放在步号、任务栈、帧栈与分支细节里[20]

6.5 架构能力从哪里来

幻灯片在十行与两千行之间插了一页反问：“这种能力从哪里来?”它给出的答案很朴素：来自你在学校的学习与社会上的积累。讲者把它展开成三层。第一层是压缩：能看出一段代码背后的规律，是因为你把它压缩成了一个更小的表示。第二层是第一性原理：从最根本的机制推起，理解递归如何被机器执行，就不必背模板。第三层是共情：能理解快退休的教务老师怎么看待一个系统，才可能做出合适的设计。他还补了一句玩笑：如果有一天这种能力可以被强化学习直接学出来，那才是真正见证人类完蛋。

6.6 本章小结

十行的汉诺塔没有唯一的正确答案，四种表达各自把复杂性安放在不同位置：步号与盘号的数学规律、待办任务的栈、函数执行的帧栈、以及搜索排名靠前的分支细节。差别不在能否运行，而在于改需求时哪些知识还能保留。下一章把尺度放大两百倍，看两千行的项目如何面对同一种压力。

7 两千行的架构：在 Excel 里生成二维码

7.1 一个“需求非常明确”的项目

讲者的第二个尺度是两千行。项目本身很具体：在 Excel 里生成二维码。需求看起来毫无歧义——输入是一个字符串单元格，输出是一个零一序列的单元格，任务完全确定。幻灯片却把它的名字写成“QR Code Generator 的复杂性”，并列出三条：公式极长且脆弱；直接写公式面临立即崩溃；需求变更极度痛苦。其中“看到 Agent 和括号不配对搏斗”是一个很形象的现象：Excel 的公式表达力有限，把整个二维码算法塞进一个（或几个）单元格，最后会得到一份人无法阅读、也无

[20] 视频画面时间区间：00:57:30–00:58:00。若为裁剪图，时间区间仍对应原视频画面。

法调试的表达式——讲者用“从国防部偷来的 LISP 程序，最后一页全是括号”的经典笑话来形容这种长度。

需求变更的痛苦来自几个具体方向。第一是版本：Excel 从 2019 到 2023 持续增加新函数，查表从前要写 `VLOOKUP`，后来微软终于受不了，推出了更顺手的 `XLOOKUP`；于是“兼容旧版本”与“使用新语法”成为两个方向，选错方向的代价是整体重写。第二是位置：如果每个单元格都硬编码坐标，把二维码整体向右移一格就会立刻崩溃。第三是计算方式与界面：换一种算法、加一个字符串单元格，都可能牵动整张表。

“需求明确”往往只是“还没碰到坑”

这个案例最容易被误判。看需求描述，它比教务系统简单得多；但把需求交给实现时，版本兼容、位置依赖、算法替换、输入输出布局全都会变成新的约束。讲者的结论是：需求看似确定，维护起来仍然极度痛苦，两千行的项目还能重做，一旦它嵌进更大的系统就只能“加钱、加人”。

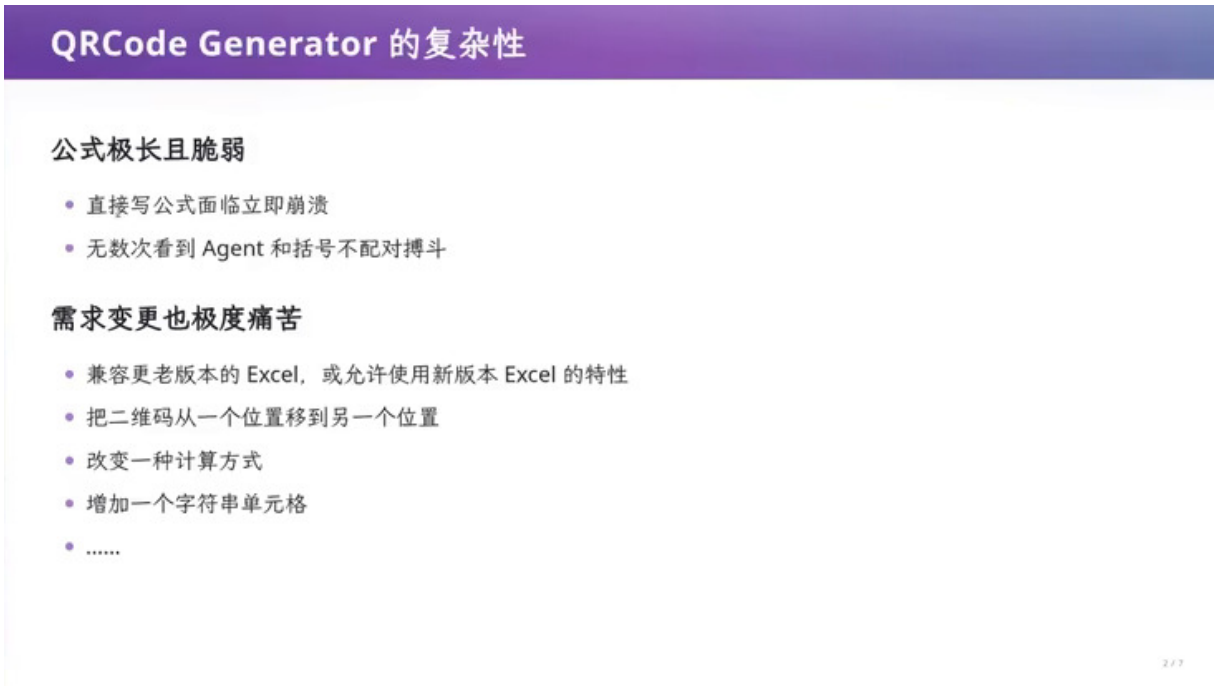


图 21: Excel 二维码生成器：公式极长且脆弱，需求变更（版本、位置、算法）都极痛苦[21]

7.2 跳出问题看问题：这其实是一个编译问题

讲者的转折点是一次视角切换。二维码算法本身容易用伪代码或编程语言描述，但没有任何办法把它原样变成 Excel 公式——不存在一个把 Python 编译成 Excel 的编译器（幻灯片甚至用“PYXLC 不存在”来点明这一点）。而语言模型的直接做法——让模型从自然语言直译到公式——已经被证明会在需求漂移时崩溃。于是真正的问题被重新表述成：在 Python 与 Excel 之间，构造一门中间语言。

[21] 视频画面时间区间：00:59:00–00:59:30。若为裁剪图，时间区间仍对应原视频画面。

中间语言：先决定表达，再决定落地

幻灯片把结论写得很干脆：二维码算法容易用伪代码或编程语言描述，但它们不能原样变成 Excel 公式；因此先用中间语言（IR）表达计算规则，再决定怎样落到 Excel 上。这一思路与 `getopt` 完全同构：先得到一个与来源无关的中间表示，再从中生成目标。讲者说这就是“在不同尺度上反复出现的同一个设计”。

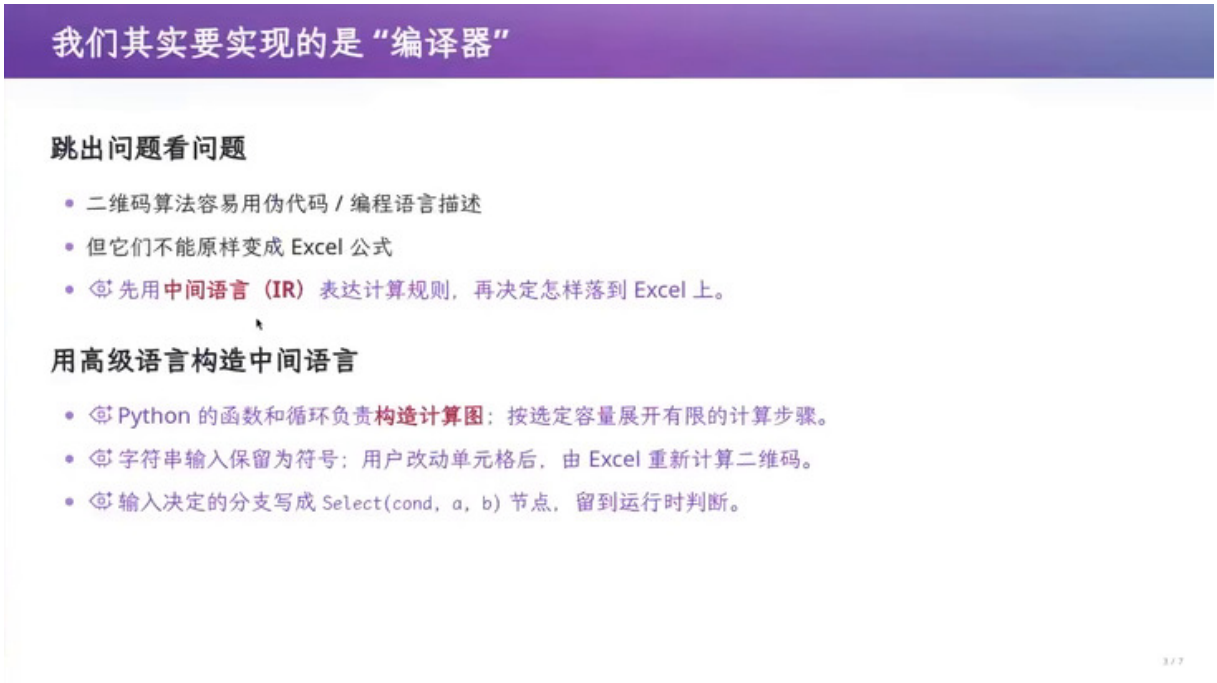


图 22: “我们其实要实现的是一个编译器”：先用中间语言表达计算规则，再落到 Excel 上[22]

7.3 中间语言长什么样

幻灯片给出了这层中间表示的三条设计要点。第一，用高级语言构造中间语言：Python 的函数和循环负责构造计算图，按选定的容量展开有限的计算步骤。第二，字符串输入保留为符号：用户改动单元格之后，由 Excel 自己重新计算，而不是在生成时把输入代入。第三，输入决定的分支写成 `Select(cond, a, b)` 节点，留到运行时判断。于是同一个计算图可以有两个后端：一个生成便于调试的 Python，一个生成 Excel 公式。

```
# 0. freeze design decisions -- this is where the difficulty lives
V, EC, MASK = 5, L, 0         # version, error correction level, mask
SIZE = 17 + 4 * V             # 37x37 modules for v5

# 1. data encoding
bits  = 0b0100                # byte mode indicator
bits += len8(s)               # character count, in bytes
bits += bytes8(s)             # each byte of the input
bits += terminator
bits  = pad_to_byte(bits)     # align to codewords
```

[22]视频画面时间区间：01:04:30–01:05:00。若为裁剪图，时间区间仍对应原视频画面。

```
bits += fill(0xEC, 0x11)      # until 108 bytes are filled (v5-L)
data  = bytes(bits)

# 2. Reed-Solomon over GF(256), primitive polynomial 0x11D
g   = rs_generator_poly(26)  # 26 = ecc codewords for v5-L
ecc = data * x**26 mod g     # division implemented with an LFSR
cw  = data + ecc             # 134 codewords in total

# 3. placement, masking, format info (omitted here)
```

Listing 5: 二维码计算的伪代码骨架（据幻灯片上的模型输出整理）

这张伪代码里出现了两个可直接引用的定量关系。第一个是尺寸：版本号 V 决定模块数，

$$\text{SIZE} = 17 + 4V$$

其中 SIZE 是二维码每边的模块数（$V = 5$ 时为 37×37），V 是版本号，取值 1 到 40。第二个是纠错编码：

$$\text{ecc} = \text{data} \cdot x^{26} \bmod g(x), \qquad g = \text{rs_generator_poly}(26)$$

其中 data 是补位后的数据码字多项式，x^{26} 表示左移 26 位，g 是 26 次生成多项式，ecc 是 26 个纠错码字；数据码字 108 个加纠错码字 26 个，合计 134 个码字。幻灯片还补充了这一节的关键：先钉死版本、纠错等级与掩码，把无穷多实现收敛成一个——因为“消灭歧义”才是真正的难点所在。

7.4 一份中间语言，两种执行方式

架构的收益在这里显形。幻灯片用“一份中间语言，两种执行方式”总结：同一个计算图既能生成便于调试的 Python 程序，也能生成 Excel 公式。生成端把符号文字保留成符号，按选定容量展开计算图，再把语义映射到单元格地址；Excel 后端则按目标版本降级、安排单元格布局、生成公式工作表。讲者的用法很实际：先用 Python 后端验证公式写得对不对，写对了再去写 Excel。

优化也找到了自己的位置。幻灯片列出四条：常量折叠——固定参数下的定位图案、有限域常量表在生成时就算好；公共子表达式——相同的纯计算在图中共用一个节点，避免把长公式反复粘贴；公式内复用可用 `LET`，跨公式共享则在目标版本不支持时退回辅助单元格；最后按依赖关系安排计算——结合复用次数、计算量与目标版本的公式限制，决定内联还是拆格。选择依据不是感觉，而是测量：分别测量首次完整计算与修改输入后的重算时间，同时观察工作簿体积，避免把缓存成本藏起来。

一个好架构会把未来的变化变成“换一个算子”

讲者强调，这套设计最实际的好处是抗变化：Excel 版本升级，无非把一些算子换一种实现；需求换了，换的是中间语言到目标后端的映射，而不是整张表。他顺带指出，AI 基础设施领域的做法与此相似——领域知识让工程师很早就做出了这类分层设计，值得学习。

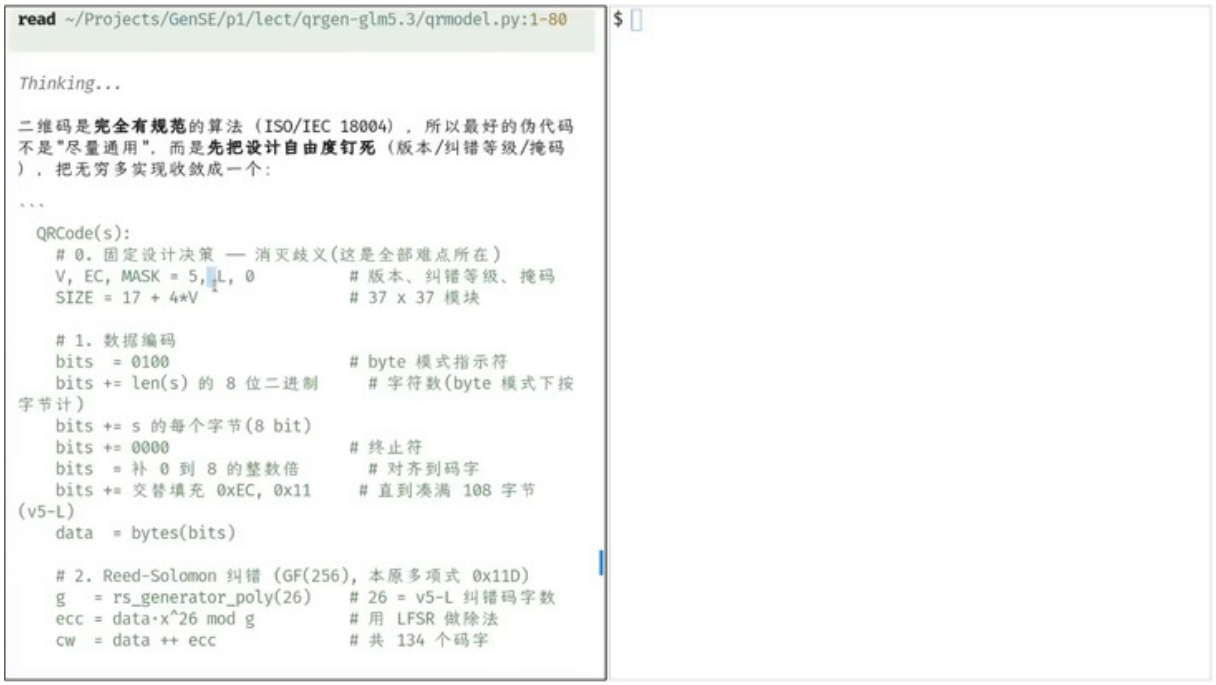


图 23: 二维码伪代码：先固定版本、纠错等级与掩码，再做数据编码与 Reed-Solomon 纠错[23]



图 24: 一份中间语言，两种执行方式：Python 后端用于验证，Excel 后端用于交付[24]

[23]视频画面时间区间：01:05:30–01:06:00。若为裁剪图，时间区间仍对应原视频画面。
[24]视频画面时间区间：01:10:30–01:11:00。若为裁剪图，时间区间仍对应原视频画面。

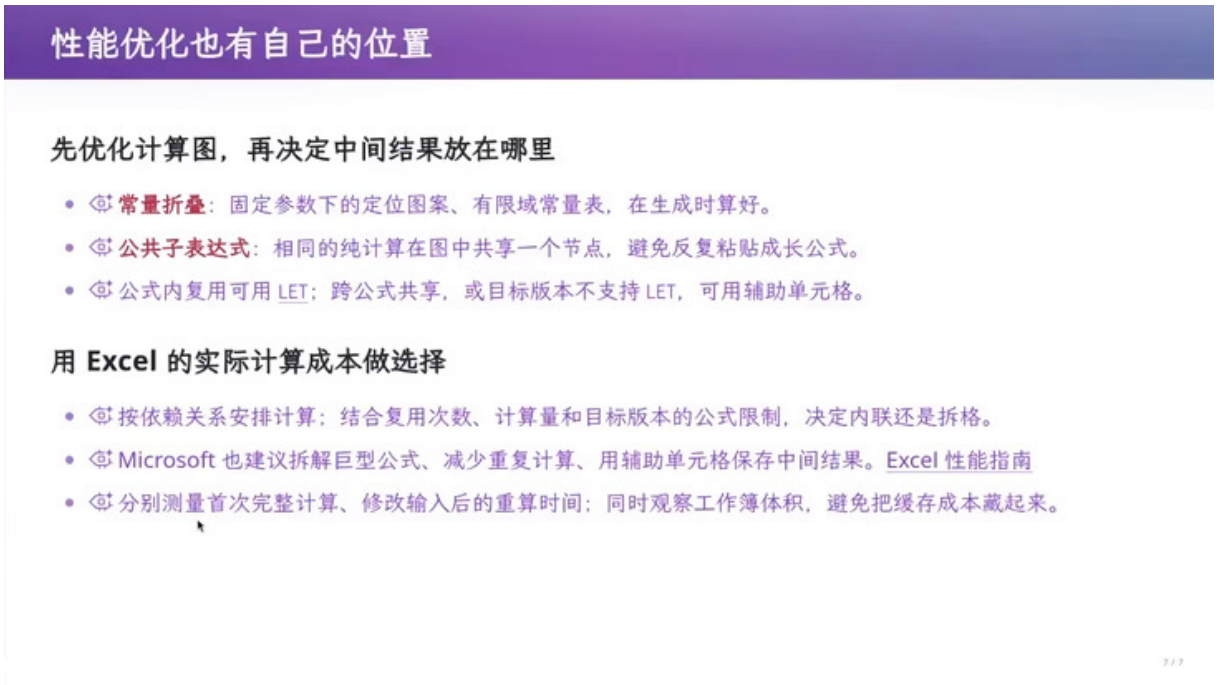


图 25: 性能优化也有自己的位置：先优化计算图，再决定中间结果放在哪里[25]

7.5 本章小结

两千行的项目给出的结论是：当需求看似明确却暗含大量变化点时，正确的动作不是把算法硬塞进目标平台，而是为“表达”留一层。中间语言让同一份计算有了两个后端，也让优化有了独立的位置。下一个尺度的项目有五十万行，而它并不是人写的——它由模型生成，需要回答的问题因此变成了：模型交出来的软件，架构上处于什么水平？

8 五十万行的架构：AI 交出来的教务系统

8.1 先用一句话问它：这是什么架构

讲者把尺度放大到五十万行，接着做了一个很有代表性的动作：不再从框图、数据库、微服务这些词出发，而是先看“现在的教务系统是怎么实现的”，然后让 AI 直接搓一个 MVP。生成结果是一个可运行的教务系统原型，体量可以从幻灯片读到：前端是 React 19 加 TypeScript 与 Vite，没有路由库、没有状态管理库，全部逻辑挤在 `src/App.tsx`（2305 行）与 `src/Services.tsx`（1177 行）两个巨石组件里，靠 `useState` 与手写的 `fetch` 封装驱动；后端是 Express 5 的单文件 `server/app.mjs`（522 行），用裸 SQL（`node:sqlite` 的 `DatabaseSync`）建表，没有 ORM、没有 migration；持久层用一张 resources 表以 JSON blob 存放“七类扩展服务”，只有课程、成绩、申请这类核心业务才用关系表。讲者在口述时给出的文件体量是 83KB 的 TSX 与 38KB 的 services，与行数互相印证。

8.2 它自己都做对了什么：生产级横切设计

更值得注意的是它自动带上的一批横切设计。幻灯片把它们称为“它最大的亮点”：乐观锁——所有写请求带 version，校验失败返回 409；幂等——前端每个写请求自动带 `Idempotency-Key`，后

[25] 视频画面时间区间：01:12:00–01:12:30。若为裁剪图，时间区间仍对应原视频画面。

端在 mutation() 里存键、身份、内容与响应，重试时去重；事务——用 BEGIN IMMEDIATE / COMMIT / ROLLBACK 把“业务写入 + 审计 + 幂等记录”包在一起；审计——audit 表记录操作者、对象与前后值；统一错误码——400、401、403、404、409。讲者对此的评价是：这些设计至少要摸爬滚打几年才理解，而模型具备这种工程直觉。

横切设计（cross-cutting design）

与“每个功能各写一遍”相对，横切设计把一类关注点收敛成一条统一管线：所有写入都经过同一个 mutation()，于是幂等、事务与审计天然生效，新接口不必重复实现。它是判断一个系统是否“长大”的重要标志。

8.3 它的扩展性好在哪里，坏在哪里

幻灯片把扩展性分成两面。好的一面是数据驱动的横向扩展：增加一类“扩展服务”很便宜，只要在 server/resources.mjs 的 resourceSpecs 里加一条字段规则，validateResource 就会自动校验，前端补一个页面即可，不用改后端逻辑；所有写操作都走同一条 mutation() 管线，新增接口天然获得幂等、事务与审计。差的一面是纵向与异构扩展：业务规则硬编码在 validateApplication 里，按业务种类写死的 if/else；加一种新申请类型，或者引入多级审批、并行审批、可配置流程，就必须同时动后端源码、状态机与前端页面；前端巨石组件没有路由、没有按域拆分，加一个页面就是往两千行文件里塞；用 JSON blob 存的资源表无法按字段查询、无法做关联与多人协作，也没有版本历史。

表 4: AI 生成系统的扩展性边界（据幻灯片上的模型自评整理）

好的一面：同类项可配置	差的一面：新流程仍需改代码
新增扩展服务只需在 resourceSpecs 加字段规则	新申请类型要改后端源码、状态机与前端页面
validateResource 自动校验，前端补页面即可	业务规则以 if/else 硬编码在 validateApplication 里
写操作共走 mutation() 管线，天然获得幂等、事务、审计	多级/并行审批与可配置流程必须改代码
——	前端巨石组件无路由、无按域拆分
——	resources 表用 JSON blob，无法按字段查询、无版本历史



图 26: AI 生成系统的架构自评：三层单体、前后端分离与生产级横切设计[26]

8.4 它还差什么：88 条需求与“横切面切片”

幻灯片里还留了另一段模型自评：README 与 implementation.md 明确列出 88 条需求中有大量“未纳入首版”；能力上没有 SSO、电子签章、消息送达、附件、政策规则库、多级审批，教室排课只“记录办件结果”，不会真的改课表或占用教室。真正的教务系统（正方、强智那一类）本质上是多角色、多学期、强一致、带可配置工作流引擎的大型领域系统。这个 demo 的价值不在“像教务系统”，而在于用极小代价把版本冲突、幂等、事务一致性、权限隔离、审计这些真问题讲清楚——它更像一个横切面切片，而不是一个系统。幻灯片还引用了模型自己的一句警告：如果不拆分、不统一规则，新增功能会越来越像拆盲盒——“能维护”与“好维护”在这里被明确区分开来。

8.5 本章小结

五十万行系统的原型说明了两件事：模型已经能自动做出相当专业的分层与横切设计；但它做出来的东西仍然是一个“当前状态”式的系统——只不过把状态换成了表与 JSON blob。要判断这种架构能活多久，需要先把它抽象成模型：软件到底是什么？

9 CRUD 架构：把世界当作当前状态的投影

9.1 一个几乎无处不在的世界观

讲者先不急着批评，而是把这种架构背后的世界观说清楚。幻灯片给出的第一条原理是：软件是物理世界过程在信息世界里的投影，而物理世界是有状态的——everything is a state machine。于是软件系统很容易被建模成状态迁移系统：任何时刻存在一个“当前状态”的快照，里面有学生、教师、选课等等。持久状态用数据库承载，可以用一张表表示学生，用另一张表表示学院、选课，字

[26] 视频画面时间区间：01:15:30–01:16:00。若为裁剪图，时间区间仍对应原视频画面。

段与表之间的关系由数据库课的范式要求来约束。前端与后端分离的经典结构，本质上就是把这份状态投影成不同的视图：今天某位同学办完退学，那就是一次 update，系统从一个状态迁移到下一个状态；你在浏览器里打开的页面，看到的是这份状态的一个投影。

CRUD 的隐含假设

“增删改查”之所以能工作，是因为它假设世界可以被压缩成“当前状态”这一份数据。只要这个假设成立，CRUD 就是一个完美架构：查询便宜、实现直接、前后端分工清晰。后面所有的麻烦，都来自这个假设在真实系统里不成立。



图 27: CRUD 架构：用“当前状态”描述整个世界，并为每个需求点写一个查询或修改[27]

9.2 范式化：把冗余从表里赶出去

数据库课要求的规范化在这里有了具体的意义。幻灯片举的例子很日常：学生表里有学号、姓名、院系等字段，学号唯一，姓名可能重名；如果另一张表要引用一个学生，尽量不要复制姓名，而是放一个学生 ID。这个 ID 相当于一个链接，指向学生的信息，学生信息又指向别的表。这样做的好处是最小化依赖：一个地方改了，不需要去改另一个地方。讲者特意强调改名在中国是合法的：把姓名存在两处，改一次就要改两个地方，代价立刻显形。这套思路与后面的架构问题是同一个母题——消除冗余、让事实只存一份，从而消除不一致的来源。

范式化的直觉：不要让同一事实有两个出处

“第二范式”“第三范式”这些术语容易让人迷失，它们要解决的问题却很朴素：同一件事被记了两遍，就一定会有两份记录不一致的那一天。用外键（ID 关联）替代复制字段，是把“同一事实”收敛到唯一出处的具体做法。

[27] 视频画面时间区间：01:16:30–01:17:00。若为裁剪图，时间区间仍对应原视频画面。

表 5: 范式化处理同一份信息的两种做法（据讲解整理）

场景	复制字段	ID 关联
另一张表要引用学生	表里再存一份姓名	只存学生 ID，指向学生表
学生改名	两处都要改，否则不一致	改一处，另存曾用名即可
重名	无法区分	学号唯一，天然可区分
信息来源	两个出处	唯一出处

9.3 本章小结

CRUD 架构的世界观是“世界＝当前状态”，范式化则保证这份状态内部尽量不矛盾。只要世界是静止的、规则是不变的，这套架构就是最优解。下一章说明为什么在教务系统里这些前提都不成立，以及失效的具体形态。

10 CRUD 为什么会死：历史、规则与决定

10.1 “当前状态”里其实混着三种东西

幻灯片给出了一句很关键的判断：当前状态里混着**事实**、**规则**与**决定**。它等于历史事实，加上当时适用的规则，再加上按规则作出的决定。问题出在大量单点数据其实是带 data dependency 的“结果”——依赖意味着前面改了会摧毁后面。而更糟的是，这些依赖关系往往没有被很好地管理起来。

讲者用“允许毕业”这个字段做例子。它究竟依赖什么？幻灯片列出的答案是：课程成绩、课程替代审批与培养方案版本，共同决定学分认定与毕业结论。这带来两个无法回避的追问：成绩更正之后，哪些结论需要重新审核，哪些已经成为正式决定？以及——表中没有重复字段，也可能丢掉“这个结论是怎么算出来的”。你知道某位同学能毕业，却不知道他为什么能毕业；你知道他不能毕业，也不知道是哪一条规则卡住了他。计算过程被丢掉了。

结果留在表里，过程留在哪里

把派生结论物化成字段（`can_graduate = true`），性能上很划算，代价是丢掉了它的推导链。一旦规则变化或数据被更正，你无法判断这个已存在的结论是否仍然成立，只能逐个重新计算，甚至重新开会讨论。

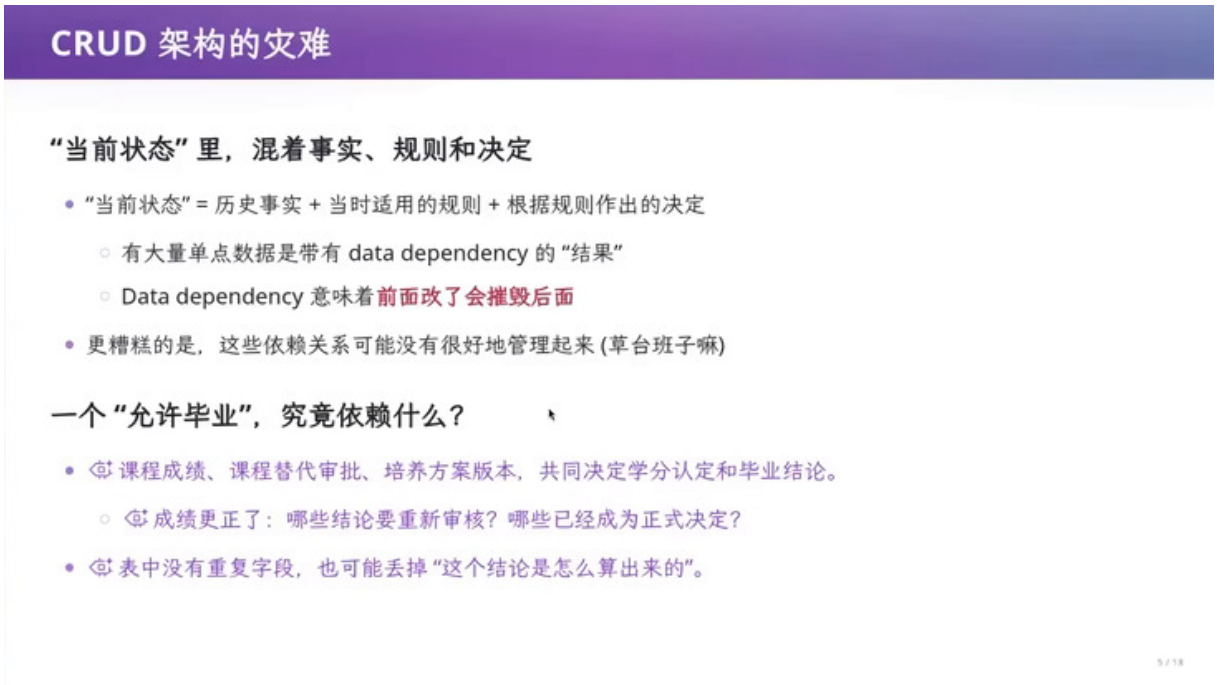


图 28: CRUD 架构的灾难：当前状态里混着事实、规则和决定，依赖关系却没被管理起来[28]

10.2 有损投影：系统外的事情已经发生了

第二类失效来自投影的有损性。幻灯片写得很有画面感：系统外，事情已经发生了——政策更新、领导批准还没有进入教务系统；而“过了今天这小朋友就不能毕业了”，于是有人赶紧在系统里把毕业条件改造成满足。讲者补充的现实版本是教室维修：物理世界已经发生（教室不能用了），系统里没有建模这一点，只能靠人工重新安排再手改回去。这样的操作累积下来，系统里就出现大量不一致的数据，随着时间变成无法解释的黑洞：明明这个同学没有交学费，为什么他能毕业？明明这个同学不该毕业，他为什么毕业了？

幻灯片给出的方向是把例外也变成可表达的业务：记录“谁批准了哪项豁免、适用于谁、依据什么”，再由审批产生毕业决定。只留下一个 `can_graduate=true`，以后连“是特批”还是“算错了”都分不清。

有损投影：这是所有信息系统的宿命

任何信息系统都只能承载物理世界的一部分。差别在于：有损的部分是被显式建模成“例外与审批”，还是被当成脏数据塞进字段里。前者可以被查询、被审计、被追责；后者会静默地污染后续所有推理。

[28] 视频画面时间区间：01:22:00–01:22:30。若为裁剪图，时间区间仍对应原视频画面。

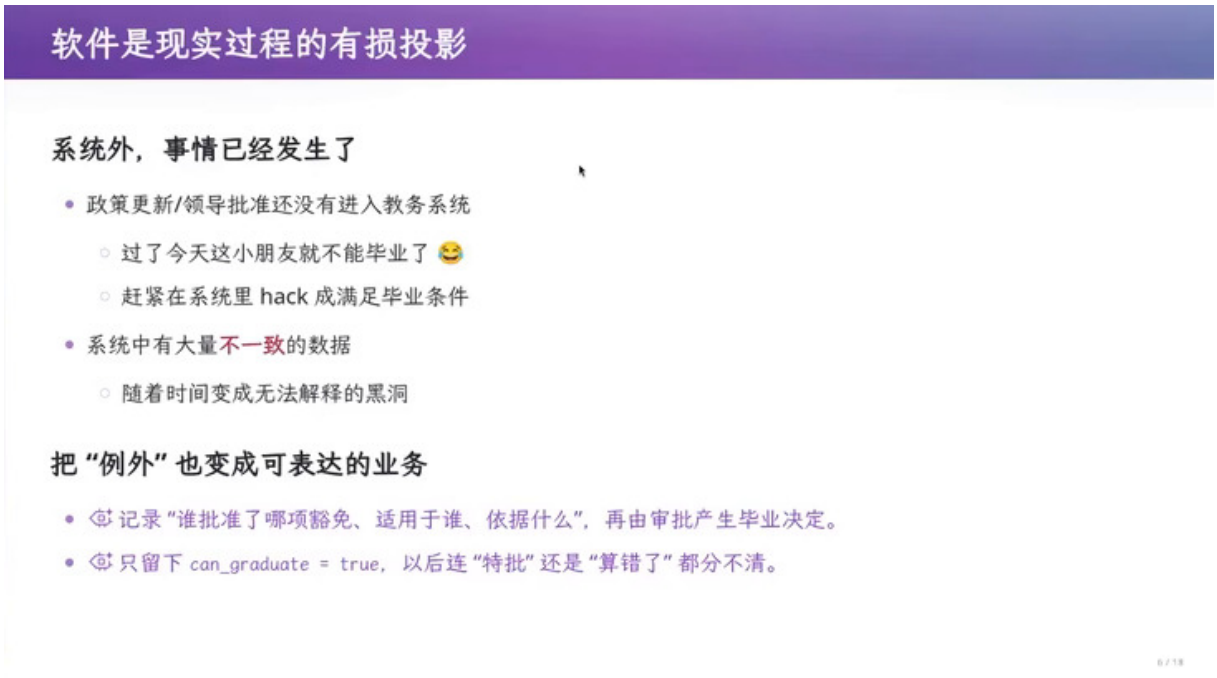


图 29: 软件是现实过程的有损投影：系统外的事情已经发生，系统内只剩一个结果字段[29]

10.3 语义也会变：计算机系变成计算机学院

第三类失效最刁钻：不是数据错了，也不是规则改了，而是**同一个词的语义变了**。幻灯片上写着“南大没系啦”——计算机系变成了计算机学院。喜大普奔，教务系统却瑟瑟发抖，因为它是所有事实的 single point of truth，于是必须回答：`dept_id` 应该沿用旧的，还是新开一个？

两个选择都在承诺一种业务语义。沿用旧 ID、覆盖名称：学生、课程、权限关联继续有效，旧报表一 JOIN 就能用；代价是历史上的“计算机系”也变成了“计算机学院”，一个 2011 年毕业的学生在系统里会被打印成“毕业于南京大学计算机学院”，与事实不符。新建 ID：旧组织得以保留；不迁关联，新学院像个空壳；全量迁关联，学生和课程过去的归属又被改写。跨年统计、权限继承、培养方案适用范围，都需要先定义旧组织与新组织的关系。

表 6: 院系调整的两种建模方式及其业务语义（据幻灯片整理）

选择	结果	代价
沿用旧 ID	学生、课程、权限关联继续有效；旧报表 JOIN 即用	历史报表里的“计算机系”被改写成“计算机学院”，与当年事实不符
新建 ID	旧组织保留，历史归属不被改写	不迁关联则新学院是空壳；全量迁移则学生与课程过去的归属被改写

幻灯片给的处理原则是：先判断身份是否延续，再给变化建模。如果业务认定组织身份延续，就保留稳定的 `org_id`，把名称、层级、隶属关系按有效期保存版本；如果业务认定产生了新组织，就建立新 ID，记录带日期的承继关系，学生与课程按业务决定转移并保留原归属；查询时要明确选择“当时归属”还是“按今天口径汇总”，培养方案单独绑定版本，不能让新名称自动触发新规则。

[29] 视频画面时间区间：01:23:00–01:23:30。若为裁剪图，时间区间仍对应原视频画面。

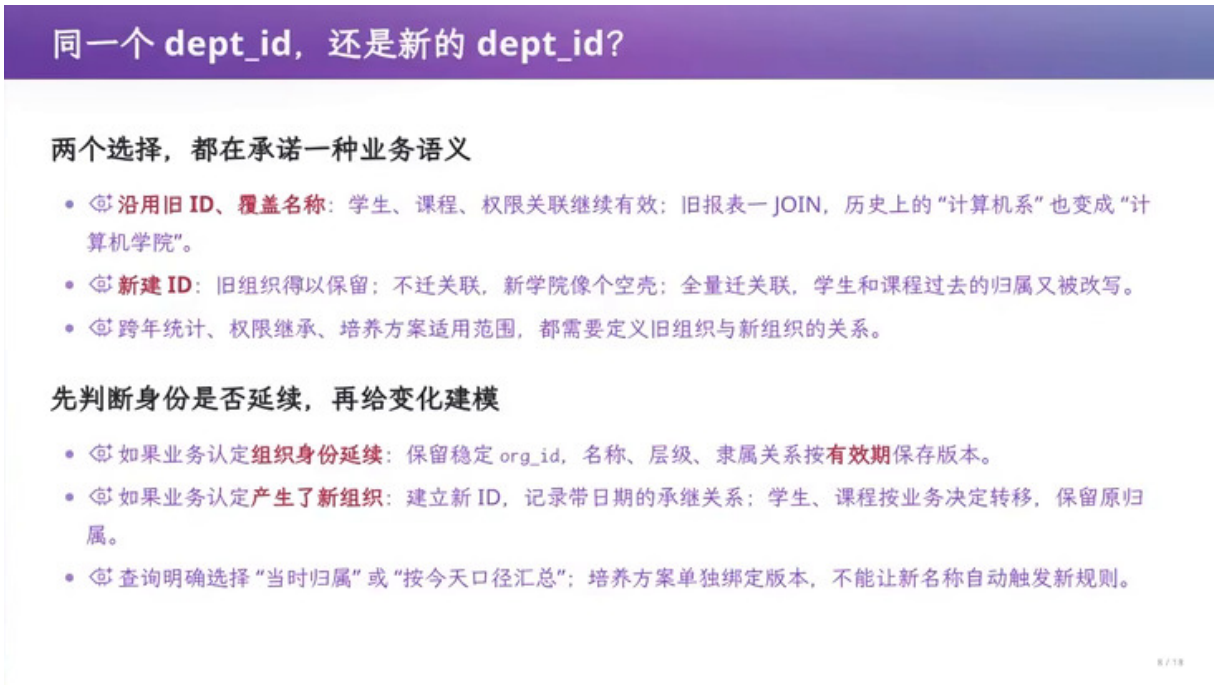


图 30: 同一个 `dept_id` 还是新的 `dept_id`：两个选择都在承诺一种业务语义[30]

10.4 系统一旦跑起来，修改的代价极高

把这三种失效放在一起，就得到讲者的结论：代码可以重写，历史不能凭空补出来。数据库迁移与业务逻辑重写都是风险极高的操作，而迁移还必须决定哪些历史含义继续成立——没有保存依据的话，AI 也推不回来。幻灯片因此说，软件的**设计、实现、维护都是手艺活**，这也是 ERP 类系统的护城河。

讲者的两个例子把代价说得很具体。第一个是测试环境与生产环境的差别：在测试环境做一次迁移，和把学校里所有真实数据做一次迁移，难度完全不一样；他自己把选课记录改成 -1 导致页面崩溃的经历就是提醒——数据库里已经存在大量不一致时，你很难写出一个规则来迁移。第二个是他读博时的见闻：隔壁软件工程组接了学校研究生院教务系统的订单，金额大约 20 万 30 万元，在定制软件里算小订单，因为要不停维护这座史山。那时他就判断这种典型 CRUD 架构会遇到麻烦；后来研究生大幅扩招、培养类型增加、管理要求频繁调整，系统最终彻底变成石山，如今统一由有领域知识的公司维护。幻灯片还提醒，这类系统的厂商往往在 2000 年前后成立，参考一个已有的企业架构切入教育市场，一旦背下这笔技术债，就要在中国大学教育改革的几十年里不断扛下所有。

“先跑起来再说”在业务系统里的代价

原型阶段的快速上线会把历史“压扁”成当前状态。等到需要回答“当年为什么这么判”时，压扁掉的过程无法恢复。迁移的风险不仅在于数据搬不动，更在于搬过去之后，新旧语义的边界没人能说清。

[30] 视频画面时间区间：01:26:00–01:26:30。若为裁剪图，时间区间仍对应原视频画面。

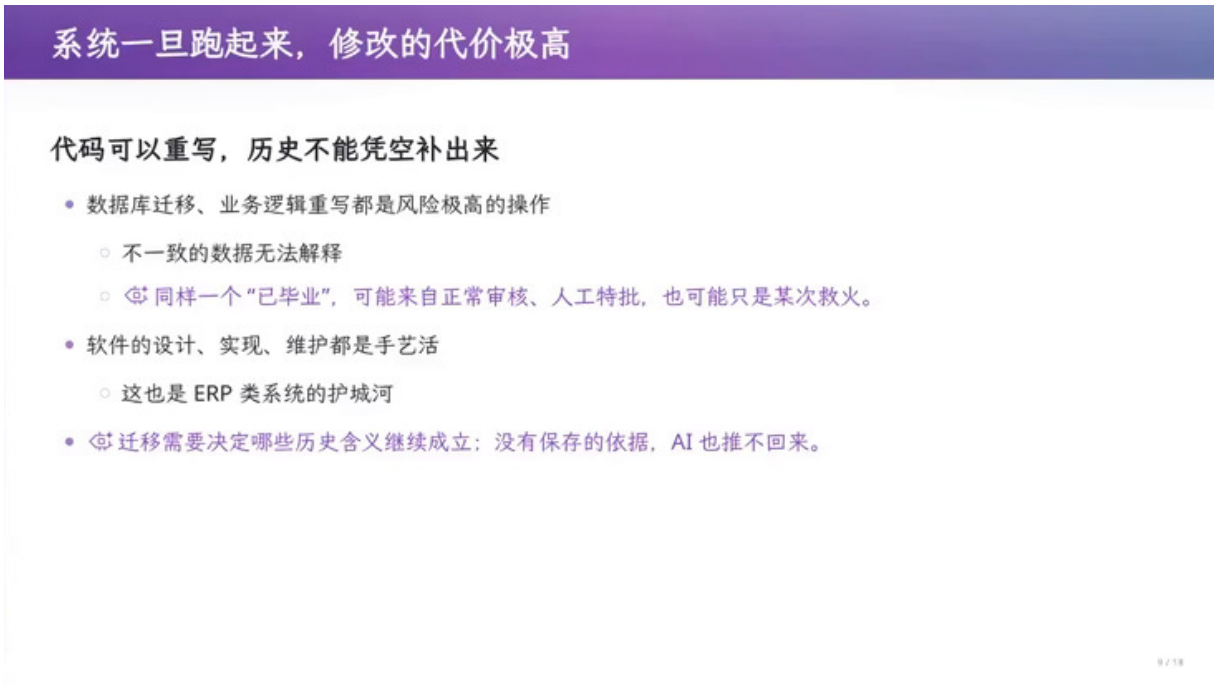


图 31: 系统一旦跑起来，修改的代价极高：代码可以重写，历史不能凭空补出来[31]

10.5 本章小结

CRUD 架构在教务系统上失效有三种形态：派生结论丢掉了推导过程；物理世界里已经发生的事没有被建模，只能靠 hack 补；甚至连“系”与“学院”这样的词，语义都会变。三者共同指向一个技术需求：**要有一个地方能保存事实与过程本身**，而不是只保存最后那一刻的汇总。下一章给出讲者的答案。

11 事件溯源：让事实成为一等公民

11.1 三种等价视图

讲者转回数据结构课上的一个概念：persistent data structure（持久化数据结构）。它可以保留这份数据结构的所有历史。幻灯片把它表述成“数据结构的两面”：数据结构既可以记录“当前状态”，也可以记录“操作历史”，而状态可以由 replay 事件序列决定——语义就是状态机。

从存储角度看，有三种视图是等价的。存当前状态，就是 CRUD；存操作日志，就是数据库与操作系统课上讲过的 write-ahead journal；存操作对状态的 delta，就是 git、SVN 这类版本管理里的思想。三者的共同点是都可以通过回放历史得到当前状态，差别在于你付出了多少存储与计算，换回了多少可解释性。

[31] 视频画面时间区间：01:27:00–01:27:30。若为裁剪图，时间区间仍对应原视频画面。

表 7: 同一份事实的三种存储视图（据讲解整理）

存什么	对应技术	代价与收益
当前状态	CRUD、关系表	查询快、写入直接；历史与推导过程消失
操作日志	write-ahead journal	保留事实顺序；需要 replay 才能得到状态
状态的 delta	git / SVN 式版本管理	便于比较与回退；需要定义 delta 的语义

11.2 用事件序列换取解释力

讲者用“张三退学”的例子说明这件事的价值。假如当初为了省事，退学就是把张三从数据库里抹掉，那么后来需要“复学”这个操作时，就得把与其他部分的关联重新建起来，代价很大。反过来，如果系统记录的是操作与事件，就可以通过 replay 知道张三复学前的状态是什么，状态没有被真正丢弃。讲者强调逻辑并不会因此减少：复学这件事以前没考虑过，仍然要为它写逻辑。变化的是**底气**——事实不可篡改，系统里的一切都是对事实的投影，任何一个判断做错了，都可以退回过去的某个快照，修好逻辑再来。

第二个例子是院系调整。如果“计算机系改为计算机学院”被建模成一个事件，那么查询过去的人会得到他过去的版本，也就知道他经历了什么：这位同学 2007 年入学、2011 年毕业，那时都是计算机系；计算机系在 2024 年改为计算机学院。你总是可以裁出一段与某个事实相关的历史，replay 出你需要的口径——既能查询过去，也能查询现在。

事件溯源（event sourcing）

从历史事件生成状态，而不是直接维护当前状态。幻灯片的总结很直白：祝贺，你发明了 event sourcing。它给出的直觉是账本——余额是结果，逐笔记录让我们知道结果从何而来。讲者把它与“事实的一等公民”联系起来：既然依赖关系是问题的根源，为什么不干脆把事实与依赖本身作为系统中最重要的数据保存下来。

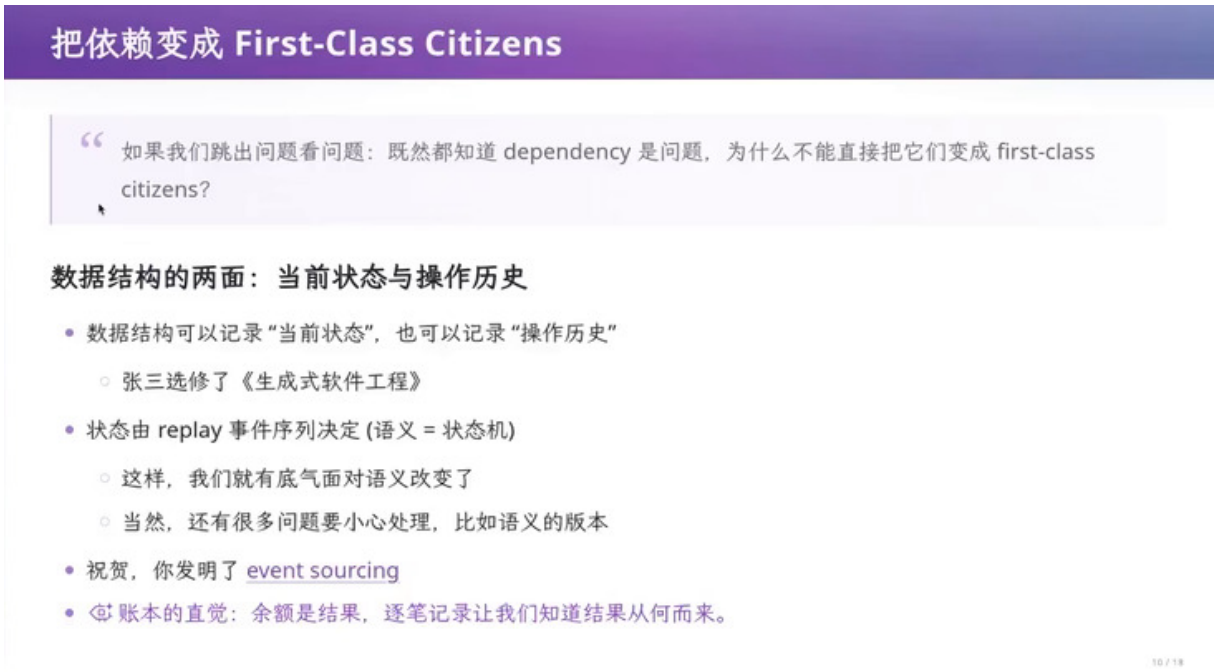


图 32: 把依赖变成 First-Class Citizens：状态由 replay 事件序列决定[32]

11.3 代价与边界

讲者没有把事件溯源说成银弹。幻灯片提醒，replay 会带来性能问题，但这不算大问题：可以用快照与缓存等手段优化。真正的困难有两个：语义的版本需要小心处理；以及**中途改造极难**——如果系统已经是一座史山，把它改成事件溯源代价巨大；而如果从一开始就按这个思路设计，很多问题根本不会发生。

这个判断恰好被视频最后一段模型自评印证。当讲者问“如果用 event sourcing 管理，会不会更好?”时，回答是分成来看，不是全局更好：这个系统里只有两个聚合是“天生事件流”的，其余是纯 CRUD，全面 ES 是拿 80% 的复杂度换 20% 的收益。它承认 ES 确实能改善三处：审计从“副日志”升为“事实来源”——现在 audit 表是一笔顺手的记录，且资源只记标题与状态，忘了写一条 event 状态与历史就会不一致，而 ES 里事件是唯一写入、状态是投影出来的，这类 bug 从结构上消失；并发冲突更自然——现在成绩更正要靠字段比对，ES 里 `append(stream, expectedVersion)` 把这件事统一成一种模式；审批状态机本就该是事件流——`draft`、`pending`、`returned`、`approved` 本来就是一条事件序列，而现在被压成了当前快照。

不要为了时髦上事件溯源

“全面 ES”会把不天生事件流的模块也拖进复杂度里。判断标准是业务本身：如果一个概念的历史与过程就是它的价值（审批、审计、账目、学籍异动），事件流是自然表示；如果它只是配置数据或统计口径，CRUD 更划算。

[32] 视频画面时间区间：01:34:30–01:35:00。若为裁剪图，时间区间仍对应原视频画面。



图 33: 模型自评 event sourcing 的取舍：只有少数聚合天生是事件流，全面 ES 是拿 80% 的复杂度换 20% 的收益[33]

11.4 术语不必先学，原理必须先懂

幻灯片最后列出几个更花哨的术语：CQRS（command query responsibility segregation）与 domain driven design。讲者提醒它们与事件溯源相互独立，可以去看、可以去学，但不必把它们当成前提。他的落点回到这一讲反复出现的方法：只要回到最初的第一性原理，思考“设计一个 50 万行教务系统时，根源的问题在哪里”，即使你从没听说过这些术语，也能做出正确的架构设计。

11.5 本章小结

事件溯源把“事实”而不是“结论”作为系统的核心：存状态、存日志、存 delta 三种视图在信息上等价，但在可解释性上不等价。它带来的收益集中在需要留痕与追溯的领域，代价则是重放成本与语义版本管理。把这条线索与前面几章连起来看，整讲的逻辑就完整了：需求的不确定性无法消灭，只能被架构吸收；而吸收的方式，取决于你选择把什么当作系统里的一等公民。

12 总结与延伸

12.1 讲者留下的结论

课程最后没有总结陈词式的收尾，只有几个判断被反复强调。代码在 AI 时代变便宜了，于是**需求**、**产品设计**、**架构**成为撬动生产力的杠杆；需求永远不可能说清楚，而且一定会变；需求的变更会穿透整个依赖图，所以“重写一遍”不是答案；架构的本质是控制复杂性，把变化安放在边界之内；当系统的核心是事实与过程时，用事件序列承载它比用当前状态更接近真相，但全面事件溯源要付出相应的复杂度。

[33]视频画面时间区间：01:36:00–01:36:30。若为裁剪图，时间区间仍对应原视频画面。

12.2 本讲义的压缩

如果要把这一讲压成一条可复用的判断链，可以这样表述：第一，需求分析把不确定的意图变成可讨论的规格，它是必要条件而不是充分条件；第二，规格一旦落地，系统就获得了它自己的历史，而历史会以数据依赖的形式约束未来；第三，架构决定了历史被保存成什么形态——是被压扁成当前状态，还是被保存成可回放的事件；第四，这个选择决定了系统在需求变化第 n 次时的维护成本。三个尺度的对照可以放在一张表里：

表 8: 三个尺度上的同一个问题：复杂性被放在哪里（据讲解与幻灯片整理）

尺度	例子	架构动作	能看到什么
10 行	非递归汉诺塔	选择中间表示（步号 / 任务栈 / 帧栈）	表示决定难度与可维护性
2000 行	Excel 二维码	引入中间语言，双后端生成	表达与目标解耦，优化有位置
50 万行	教务系统	用事件序列承载事实，而不是只存当前状态	语义变化与审计可被吸收

12.3 边界与下一步

这一讲给出的答案有明确的适用边界。事件溯源擅长“留痕即合规”的领域，却不适合纯粹的配置与统计；中间语言解决表达与目标的解耦，但要求你先把设计自由度钉死；AI 能承担需求整理、原型实现与横切设计，但它不会自动补上日志、边界校验与历史建模——这些仍需人来决定。把这三条边界记住，比记住任何一个术语都有用。

如果有下一步，讲者给出的方向是继续问下去：语义的版本该怎么管理？一个已经跑了十年的系统，能不能在不推倒重来的前提下，逐步把“事实”从“当前状态”里拆出来？这类问题没有现成答案，但这一讲提供了一条可操作的路径——先判断根源问题，再选择架构，而不是先选技术栈。

12.4 拓展阅读（幻灯片标注的出处）

视频幻灯片在讨论中标注了若干文献，供进一步阅读：Brooks 关于“没有银弹”的论述（1987）；Parnas 关于把容易变化的设计决策藏进模块内部的思想（1972）；城市规划领域提出 Wicked Problems 的 Rittel & Webber（1973）；以及关于同构变体任务难度的 Zhang & Norman（1994）。本讲义对这些文献的引用方式与视频一致：只作为幻灯片标注的出处，不对其内容做超出视频范围的解读。